

Using $f_x(n)$ in place of P_n in (10.4), the average symbol entropy in this case is

$$H_x = -\sum_{n=0}^7 f_x(n) \log_2 f_x(n) = 2.8553 \text{ bits/symbol} \quad (10.6)$$

Because the entropy in bits per symbol is less than three, the number of bits per symbol required for the ordinary binary codes (000 through 111), we know that *entropy coding*, which is discussed in the next section, may provide a way to store the signal with fewer than $32(3) = 96$ bits.

Second, we notice in the amplitude spectrum of x , which is also plotted in Figure 10.2, that not all *spectral* elements are equal. As we noted in Chapter 7, Section 7.5, this implies that the signal is *correlated*, that is, the autocorrelation function, $\phi_{xx}(m)$, is not just an impulse at $m = 0$. In other words, the samples of x are not independent, and therefore, *linear predictive coding* as it is described in Chapter 8, Section 8.10, could also be applied to compress the waveform. In Figure 10.3, we have a plot of the original vector, x , with the discrete prediction error vector, e . In this case, the symbol frequencies, similar to (10.5), are as follows:

Symbol (n) in e :	-1	0	1
Frequency ($f_e(n)$):	$\frac{11}{32}$	$\frac{6}{32}$	$\frac{15}{32}$

(10.7)

The corresponding entropy, H_e , is less than H_x in (10.6):

$$H_e = -\sum_{n=0}^2 f_e(n) \log_2 f_e(n) = 1.4948 \text{ bits/symbol} \quad (10.8)$$

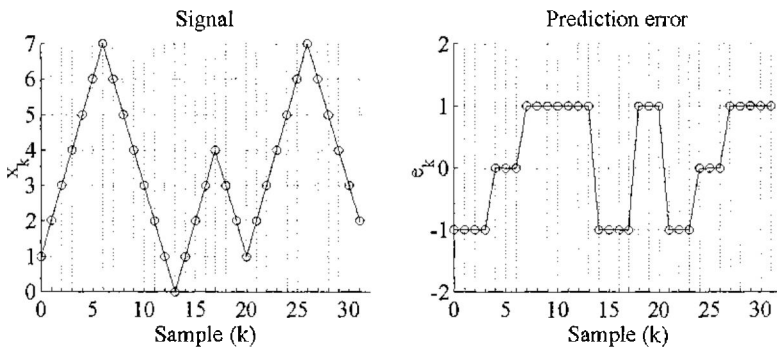
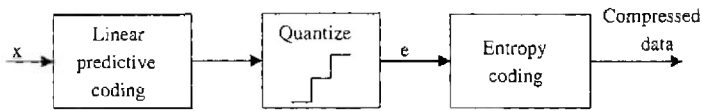


FIGURE 10.3

The signal vector in Figure 10.2 and its prediction error rounded to the nearest integer. The one-step predictor had two weights derived using covariance functions.

**FIGURE 10.4**

Compression of a signal (x) in two stages. The first stage reduces intersymbol dependence, and the second stage applies coding to reduce the number of bits per symbol in e , the quantized prediction error.

Thus, in comparison with (10.6), further compression is possible due to the lowered signal entropy. Note that the predictor weights (two weights in this case) must be stored with the error vector as described in Chapter 8, but this would not be significant with a long signal vector.

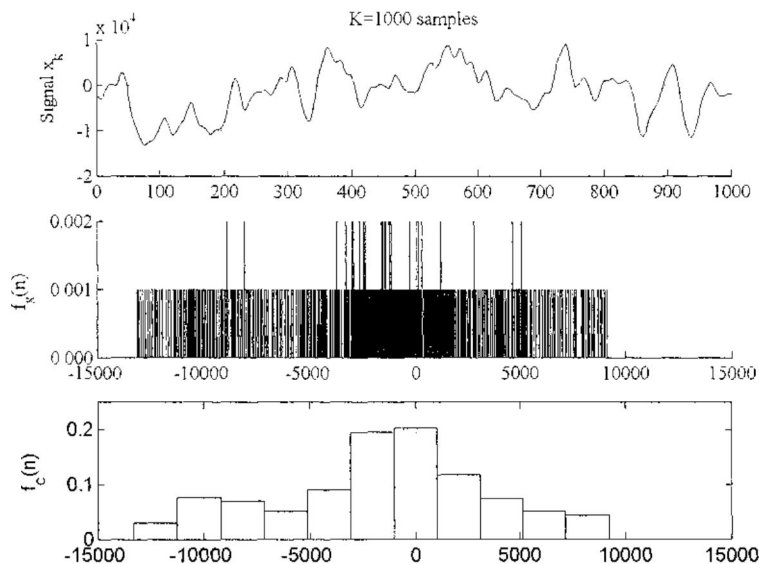
Thus, we see that there are two ways to compress a signal. One aims at encoding the signal in a way such that the number of bits is close to the signal entropy times the signal length. The second aims at reducing the range of the signal amplitude, and thus the number of bits per symbol, by removing intersymbol dependence and whitening the spectrum of the signal.

Regarding the second method, predictive coding is not the only way to reduce intersymbol dependence. Other methods include *string coding*,^{5,8-10} which is particularly applicable to text and certain kinds of images, *vector quantization*,¹¹ and some types of *transform coding*.⁷ Transform coding methods are described in Sections 10.5 through 10.7.

A final important point is that the two methods may be used *together* in a single compression operation. In the example we have been considering, a symbol code could be applied to vector e to replace the binary code of two bits per symbol and approach the entropy limit in (10.8), and we would have the scheme shown in Figure 10.4, that is, decorrelation followed by entropy coding of the quantized prediction error. In the illustration we have given here, the prediction error is simply rounded to the nearest integer, which allows exact recovery of the original vector as demonstrated in Chapter 8. One could produce further compression by increasing or varying the size of the quantizer steps, but then exact recovery would no longer be possible. This is known as *lossy compression*, as opposed to *lossless compression*, and is applicable in situations such as narrow-band telephony, or image and video compression, where exact recovery is not a requirement.

10.4 Entropy Coding

The term *entropy coding* refers to the first kind of compression we discussed in the previous section, where we encode a vector or image in a way such that the number of bits in the encoded version approaches the average entropy in bits/symbol times the number of symbols. In this section, we

**FIGURE 10.5**

(Upper) A segment, x , of the seismic waveform in Figure 8.23. (Center) Amplitude distribution of x , showing that entropy coding will not work. (Lower) Amplitude distribution of categories, which may be encoded as described in the text.

look at two coding schemes that are practical and also come close to being optimal in the sense just described.

Entropy coding begins with a set of symbols such as the eight possible waveform values in Figures 10.1 through 10.3. The coding schemes work well when the number of symbols is small but are not directly applicable when the symbol set is large. Consider the example in Figure 10.5. The upper plot is a segment consisting of 1000 samples of the seismic signal in Figure 8.23, and we note that the range of symbol values in this segment is over 20,000. The distribution of signal amplitudes in the segment is shown in the center plot of Figure 10.5. Entropy coding affords essentially no improvement in this case, because even the most frequent samples appear only twice (with frequency 0.002) in the entire segment. This leads us to the lower plot, in which $f_c(n)$ is the frequency of the n th category of symbols, each category having a range of 2048. Entropy coding then consists of encoding the category sequence rather than the sample sequence and, for each symbol, appending an 11-bit offset to locate the symbol within the category. Using this approach, entropy coding is applicable to nearly all kinds of waveforms and images.¹⁵

The first coding scheme is called *Huffman coding* after D.A. Huffman.^{2,5,12} Huffman coding is a type of *fixed-symbol coding*, in which a specific code is

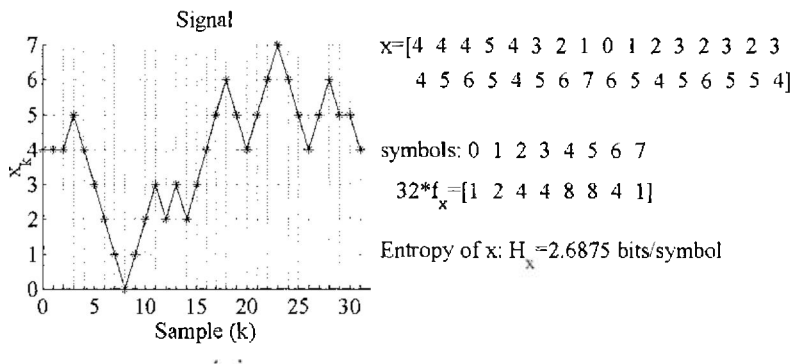


FIGURE 10.6
One of the signal vectors in Figure 10.1.

assigned to each possible symbol. Our intent here is only to give a simple example of Huffman coding that shows how to construct Huffman codes. The formal theoretical background is left to texts on coding and information theory,^{2,5} which are recommended for further study.

For our illustration, we use the second waveform in Figure 10.1, that is, the waveform shown in Figure 10.6. Next to the waveform plot in Figure 10.6 are the waveform vector (x), the frequency vector multiplied by $K=32$ (which shows the number of times each symbol appears in x), and the average entropy in bits/symbol computed using (10.4) with $P_k = f_x(k)$.

The rules for Huffman code production are simple. They involve the production of a *binary code tree*, the structure of which contains the codes for all the possible symbols. An algorithm that produces a Huffman code tree is shown next, along with an illustration of its use with the data in Figure 10.6:

HUFFMAN CODE PRODUCTION

Assign a symbol and its frequency in terms of (symbol, frequency) to each of N nodes. Then,

1. List the N nodes in order of decreasing frequency.
2. Combine nodes N and $N - 1$ into a new node.
3. Record the link from these nodes to new node.
4. There are now $N - 1$ nodes, so decrease N by one.
5. Repeat steps 1–4 until $N = 1$.

Tree information is now complete.

(10.9)

Initial	Iteration							
	1	2	3	4	5	6	7	8
(0,1)	(4,8)	(4,8)	(4,8)	(10,8)	(11,8)	(12,16)	(13,16) ₁₄	(14,32)
(1,2)	(5,8)	(5,8)	(5,8)	(4,8)	(10,8)	(11,8) ₁₃	(12,16) ₁₄	
(2,4)	(2,4)	(2,4)	(9,4)	(5,8)	(4,8) ₁₂	(10,8) ₁₃		
(3,4)	(3,4)	(3,4)	(2,4)	(9,4) ₁₁	(5,8) ₁₂			
(4,8)	(6,4)	(6,4)	(3,4) ₁₀	(2,4) ₁₁				
(5,8)	(1,2)	(8,2) ₉	(6,4) ₁₀					
(6,4)	(0,1) ₈	(1,2) ₉						
(7,1)	(7,1) ₈							

(10.10)

Initially, in the first column in (10.10), each symbol is assigned to a *node*, and the nodes are listed with corresponding symbol frequencies as shown. For convenience, we use $32f_x$ instead of f_x , so the frequencies in (10.10) sum to 32 instead of one.

Tree production begins in the second column (column 1), where the two least-frequent nodes, 0 and 7, are attached to node 8. The attachment is denoted by the subscript (8) on these two nodes in column 1, thus satisfying step 3 in the algorithm (10.9). In this manner, we move from column 1 to the second iteration in column 2, which has $N - 1 = 7$ nodes, and so on, until after eight iterations, there is a single node, which represents the top of the code tree. In the fourth iteration, notice that step 1 is necessary, that is, node 10 is not at the bottom of column 4, because there are less-frequent nodes.

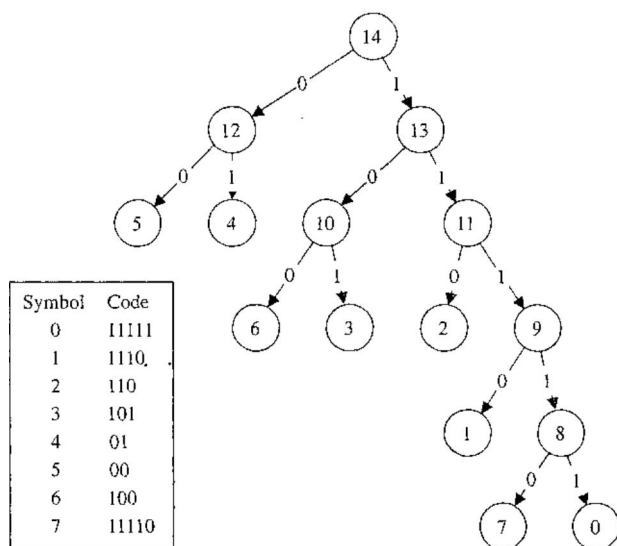
All the Huffman code information for the signal in Figure 10.6 is contained in (10.10), but it is best illustrated in the drawing of the tree in Figure 10.7. The tree is formed by connecting the nodes as indicated in (10.10), and then the codes are formed assigning binary labels (arbitrarily) to the branches emerging from each node. The resulting Huffman codes are listed in the figure.

The first thing to note about codes generated in this manner is the assignment of *shorter codes to more-frequent symbols*, which of course will lead to a shorter representation of the waveform in Figure 10.6. Second, the tree structure implies an important property called the *prefix property*, without which the codes would be useless for signal compression:

Prefix property: No code begins with another code (10.11)

We can easily see why Huffman codes have this property. If, say, 100 were the prefix of the code for any symbol besides 6, then node 6 would have emerging branches, which is impossible, because node 6 represents a symbol and not an internal node in the tree.

The prefix property implies that Huffman codes can be concatenated (strung together) to represent the complete signal vector in Figure 10.6, thereby producing a representation shorter than the original 96-bit binary representation, and yet allowing recovery of the original data. The recovery

**FIGURE 10.7**

Huffman codes and tree for the vector in Figure 10.6.

process consists (in principal) of beginning with the first bit of the binary string and proceeding down from the top of the tree (node 14 in Figure 10.7) until a symbol node is reached, recording the symbol, and repeating the process until all symbols are decoded.

The binary and Huffman encoded bit streams for the vector in Figure 10.6 are as follows, the Huffman string being encoded with the codes in Figure 10.7:

```

symbol: 4 4 4 5 4 3 2 1 0 1 2 3 2 3 2 3
binary: 100100100101100011010001000001010011010011010011
symbol: 4 5 6 5 4 5 6 7 6 5 4 5 6 5 5 4
binary: 100101110101100101110111110101100101110101101100
Huffman: 010101000110111011101111111101101011101011101010
          1001000001001001111010000010010000001

```

(10.12)

Note how the Huffman string would be decoded without ambiguity using the tree or the table in Figure 10.7.

Due to the small number (32) of samples, which is not realistic, the Huffman vector with 86 bits is only 10 bits shorter than the binary vector with 96 bits, but it illustrates the principal. Furthermore, the Huffman vector in this case is optimal. It matches the entropy of the waveform vector in Figure 10.6, that is,

$$\text{Total entropy} = 32H_x = 32(2.6875) = 86 \text{ bits} \quad (10.13)$$

This fortuitous result is not true in general. It is true, as one may observe from the nature of the binary tree, only when the symbol frequencies are all negative powers of two. Otherwise, some of the possible codes in the tree will not be used. See Exercise 7, for example, which provides such a case. The type of entropy coding discussed next gets around this constraint on the symbol probabilities.

Two final points on Huffman coding: first, there is not just one unique set of Huffman codes for a given signal, that is, for a given set of symbols and symbol frequencies. This is easily seen in Figure 10.7, where we could exchange the “0” and “1” emerging from any node and change the code without changing any of the code lengths, thus producing a comparable but different Huffman code. In fact, given an ordered set of symbols and a generally understood rule for assigning zeros and ones to branches, we could construct the code tree knowing only the lengths of the symbol codes. Thus, in Huffman coding applications, *the only data we must store or communicate with the encoded signal vector consists of the ordered set of code lengths*. Because of this, Huffman coding is used widely in signal coding and compression.

Second, MATLAB functions for Huffman coding are available from many sources, including the Internet at the time of this writing. A simple function included with this text that generates Huffman codes is

$$[H,L] = h_codes(x) \quad (10.14)$$

Given a signal x , H is a vector of Huffman codes for the symbols in x , and L is a vector giving the length of each code. (Note that L is necessary, because some codes begin with zero.) The signal vector is assumed to consist of integers in the range $[0, x_{\max}]$. The function calls two other functions. One is *code_length(x)*, which does as its name implies, and the other is *freq(x)*, which computes a symbol frequency table. The latter is useful for computing amplitude distributions in general. Its description is

$$[f, xmin, xmax] = freq(x) \quad (10.15)$$

When *freq* is executed, f becomes a vector of symbol frequencies, that is, the amplitude distribution of x , in the range $[xmin, xmax]$.

The next type of entropy coding we discuss is called *arithmetic coding*.^{5,13–15} Arithmetic coding differs from Huffman coding in that the encoded version of the signal vector or array does not consist of a sequence of codes for individual signal elements (symbols). In arithmetic coding, the signal is encoded by processing the symbols one at a time in order, and the end result is a single long binary fraction rather than a sequence of symbol codes.

To illustrate the production of an arithmetic code, we again use the waveform vector in Figure 10.6, which is shown again in Figure 10.8 with its table of symbol frequencies. The frequencies sum to one and represent the symbol probabilities for this particular vector.

The concept (but not exactly the method, as we shall see) of arithmetic coding is illustrated in Figure 10.9. The encoding process begins by establishing a line

from zero to (in this case) $2^{15} = 32,768$, with eight partitions with lengths that are proportional to the eight symbol frequencies. This is the vertical line at the left of the diagram. Notice that any one of the eight possible symbols specifies a unique piece of the line, with the length of the piece being proportional to the symbol frequency. Thus, for example, the interval from 27,648 to 31,744 stands for the symbol 6, and so on. Now instead of the entire interval, we agree (arbitrarily) to represent the interval by its *lower bound*. Thus, the symbol 6 would be represented by 27,648, and so on.

The idea in arithmetic coding is to represent the entire message as a small interval on this line. The encoding process begins in this illustration with the interval $[0, 32768]$ and proceeds with the definition of an interval that grows smaller with each symbol. As the sequence of symbols increases in length, it requires a diminishing fraction of the line to represent it, that is, to distinguish it from other sequences of the same length.

The process is illustrated in Figure 10.9, which shows the encoding of just the first four symbols in the vector in Figure 10.8, that is, $[4, 4, 4, 5, 4]$. With the first symbol (4), the range is reduced from $[0, 32768]$ to the interval $[11264, 19456]$, which is represented by the second line in from the left. With the second symbol (4 again), the interval is reduced to $[14080, 16128]$. With the third (4), the interval is reduced again to $[14784, 15296]$. The next symbol (5) selects a different part of the line and reduces the interval to $[15088, 15216]$. Notice that the number of significant bits required to distinguish the upper and lower limits of the interval (and thus the final symbol from other symbols) has increased at each stage. The final symbol in Figure 10.9 (4) causes the interval to shrink again to $[15132, 15164]$, and thus, the number of bits needed to distinguish the interval grows with each symbol.

In fact, we can see in the example of Figure 10.9 that to process all 32 symbols in the vector and still represent the intervals with integers, we would have to begin with a range much larger than 2^{15} . Because the total signal entropy is 86 bits, we would require an initial integer range of $2^{86} \approx 10^{26}$ for the entire signal. However, the range is not necessary, because the most significant bits of the interval limits can be stripped (and saved) as encoding proceeds. For example, in Figure 10.9, the binary representations of the range limits on the third line are

$$\begin{aligned}(14080)_{10} &= (011011100000000)_2 \\ (16128)_{10} &= (011111100000000)_2\end{aligned}$$

We could represent the interval just as well without the two leading bits in each binary number. Practical arithmetic coding algorithms work by stripping and saving these unnecessary leading bits as symbols are processed, thus maintaining a practical integer word size. MATLAB arithmetic encoding and decoding functions that use this technique are included with this text:

$$\begin{aligned}[y, Nbits] &= a_encode(x) \\ x &= a_decode(y, f, K)\end{aligned}\tag{10.16}$$

The functions are useful with the exercises, but probably will need modification for other applications. The encoding function, *a_encode*, encodes an integer vector or array, *x*, into another vector, *y*, with 8 bits per element of *y*, and also supplies the total number of output bits. If *a_encode* is used with the signal in Figure 10.8, the output bit stream is

Arithmetic coding of signal in Figure 10.8:

```
01110110 01000110 11000001 01011010 10110011 11100111
01010100 11011000 01001111 11011110 011101
```

(10.17)

Comparing this with (10.12), we see that the bit stream length (86 bits) is the same as the Huffman length in this example, but that there are no individual symbol codes in this case. Instead, there is a single code representing the entire signal.

The *a_encode* function makes use internally of the symbol frequencies computed with the *freq* function described previously. The frequency data must be included with the code in (10.17), or in some way made available to the decoder. The decoding function, *a_decode*, uses the encoding algorithm; that is, enough bits are read to discern the first symbol, which in turn is used to compute the interval for the second symbol just as in encoding, and so on. The arguments *f* and *K* of *a_decode* in (10.16) are the frequency table and the length of *x*, respectively.

As we have seen, Huffman and arithmetic encoding both result in an encoded vector length that approaches or matches the average symbol entropy times the number of symbols. Both encoding methods also have the overhead associated with including the symbol lengths (Huffman) or symbol frequencies (arithmetic).

Our final point regarding these entropy coding methods is that either method can be made adaptive. *Adaptive arithmetic coding* is especially easy, assuming that the range of symbols is known globally. We begin with the assumption that all symbol probabilities are equal, or with any similar globally understood assumption about the symbol frequencies. After the first symbol, say x_1 , is encoded with this assumption, the frequency table is modified by incrementing the frequency of x_1 , and so on. As encoding proceeds, in general, the frequencies become more accurate, and thus, the entropy limit is approached.

Adaptive coding has two advantages. First, it eliminates the overhead. There is no need to include code lengths or frequencies with the encoded signal. Second, in cases where the symbol frequencies change in a long signal, the encoder can adapt to these changes by continuously modifying the frequency table.

As we leave this section, we note that the entropy limit, that is, bits per symbol times number of symbols, is not usually the limit of compressibility. We saw in the previous section that there are two kinds of redundancy in

signals usually encountered in engineering, and entropy coding addresses only one of these. In Exercise 24, we apply the process in Figure 10.4 to the signal in Figure 10.8 and find that the encoded message is thereby reduced from the previous entropy limit (86 bits) to a lower entropy limit (24 bits).

10.5 Transform Coding and the Discrete Cosine Transform

In Section 10.3, we discussed the distinction between *lossless* and *lossy* signal compression. Lossy compression is appropriate in applications where some kinds of errors in the reconstructed signal do not matter to the end user. In speech communications, for example, audio signals are routinely encoded in a lossy manner in order to conserve bandwidth.¹⁰ In image and video processing, the JPEG and MPEG standards⁷ involve lossy compression techniques. These also involve a form of encoding known as *transform coding*.^{7,18}

The objective in transform coding is to transform the signal vector or array into another domain, such as the frequency domain, in which most of the essential information in the signal is contained in a relatively small part of the transform domain, that is, in a relatively few elements of the transform vector or array. Almost any kind of transform that provides mapping of vectors or arrays to another domain and allows recovery via an inverse transform is a possible candidate for accomplishing this objective, depending on the data being transformed. In this section, we discuss the *Discrete Cosine Transform* (DCT), which has turned out to be especially useful in waveform and image compression. In the next section, we discuss *multirate processing*, which produces transforms used for similar purposes.

These transforms are applied to signal compression essentially in the manner shown in Figure 10.4; that is, transform coding followed by entropy coding. The concept is shown with more detail in Figure 10.10. The signal is processed in segments, with each segment being transformed independently. In current image processing practice, for example, 8×8 pixel segments are

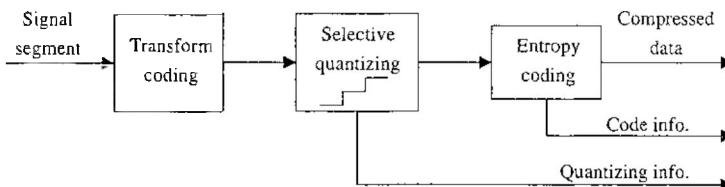


FIGURE 10.10

Signal compression using transform coding. Depending on relative importance, transform components may be quantized with different quantizing step sizes. The step sizes, along with the information needed for entropy decoding, must be included with the compressed signal data.

processed, one at a time, until the entire image is processed. Then the transform components are (or may be) quantized selectively, so that information in the more important components is kept in more detail. The definition of "more important" is both signal-dependent and subjective, and therefore, the selective quantizing scheme that works best for all waveforms or all images does not exist. Different kinds of data, and different uses of data, require different designs of this stage of the operation.

Next, in Figure 10.10, the quantized transform data is encoded, and other essential data, such as the Huffman code lengths or the arithmetic code frequencies, are attached to the compressed data. Unless it is known globally, the quantizing information, that is, the number of steps used to quantize each transform value, must also be contained in the compressed data.

Transform coding and predictive coding are competing candidates for waveform and image compression in the sense that both remove the same kinds of redundancy in a signal. For example, a predictor reduces the information in narrow-band waveform components by subtraction, and a transform coder reduces the information in these same components by concentrating them into fewer transform components.

With this idea in mind, it is natural to think first of the DFT as a candidate for transform coding. The *Discrete Cosine Transform* (DCT)^{7,17,18} is related to the DFT, and in fact, may be written as a function of the DFT, as we shall see. The DCT and the DFT are similar, because they transform a signal component at a given frequency into essentially the same transform component, and also because both transforms have unique, nearly symmetric inverses. However, unlike the DFT, the DCT has real components, which are easier to use and quantize.

The one-dimensional DCT of a vector x with N elements is defined as follows:

Discrete Cosine Transform (DCT):

$$X_m^{\text{DCT}} = c_m \sum_{n=0}^{N-1} x_n \cos\left(\frac{(2n+1)m\pi}{2N}\right); \quad m = 0, 1, \dots, N-1$$

$$c_m = \begin{cases} \sqrt{1/N}; & m = 0 \\ \sqrt{2/N}; & m > 0 \end{cases}$$

(10.18)

We use X_m^{DCT} to represent the DCT here and distinguish the DCT from the DFT. There are variations of the form in (10.18),^{7,17,18} but this form is used especially in compression, because, as we shall see, the scaling factors (c_m) produce transform components generally in a range of values similar to that of the signal components. Note that unlike the DFT, the DCT produces real components, which may be positive or negative, but not complex.

As with the DFT, there is a unique inverse DCT that recovers the original vector or array. When the DCT is expressed as in (10.18), the inverse has the

following symmetric form:

$$\begin{array}{c} \text{Inverse DCT:} \\ x_n = \sum_{m=0}^{N-1} c_m X_m^{\text{DCT}} \cos\left(\frac{(2n+1)m\pi}{2N}\right); \quad n = 0, 1, \dots, N-1 \\ c_m = \begin{cases} \sqrt{1/N}; & m = 0 \\ \sqrt{2/N}; & m > 0 \end{cases} \end{array} \quad (10.19)$$

Because the scaling factors are the same in (10.18) and (10.19), we conclude that a signal and transform elements will have comparable ranges of values. The demonstration that (10.19) is the inverse of (10.18) is similar to demonstration (3.22) for the DFT inverse and is left for Exercise 25 at the end of this chapter.

In fact, the DCT may be expressed as a function of the DFT. We do this by constructing a vector consisting of any signal vector, x , appended to itself. To begin, we use X_m^{DFT} to represent the DFT of x as defined in Chapter 3, (3.7); that is,

$$X_m^{\text{DFT}} = \sum_{n=0}^{N-1} x_n e^{j \frac{2\pi n m}{N}}; \quad m = 0, 1, \dots, N-1 \quad (10.20)$$

Now X_m^{DFT} is in general complex. But, we have seen [Chapter 8, (8.22) for example] that if x is reversed, the DFT of x' is the conjugate of X_m^{DFT} . Therefore, we should be able to combine x and x' and produce a vector with imaginary DFT components that cancel; that is, a vector with a real DFT. One such vector is the following vector with length $2N$:

$$u = [u_0 \dots u_{2N-1}] = [x_{N-1} \dots x_1 \ x_0 \ x_0 \ x_1 \dots x_{N-1}] \quad (10.21)$$

The DFT of u is then

$$\begin{aligned} U_m^{\text{DFT}} &= \sum_{n=0}^{2N-1} u_n e^{-j \frac{2\pi m n}{2N}} \\ &= \sum_{k=N-1}^0 x_k e^{-j \frac{2\pi m (k-N+1)}{2N}} + \sum_{i=0}^{N-1} x_i e^{-j \frac{2\pi m (i+N)}{2N}}; \quad m = 0, 1, \dots, N-1 \end{aligned} \quad (10.22)$$

(In the final sums, the indices are $k = N - n - 1$ and $i = n - N$.)

To satisfy the conjugate relationship between vector x and its reversal, u must now be centered at the origin, that is, halfway between the x_0 samples

in (10.21). Thus, we imagine a discrete domain in which the indices are aligned as follows:

$$\begin{array}{ccccccc} \text{element of } u: & x_{N-1} & \dots & x_0 & x_0 & \dots & x_{N-1} \\ \text{index:} & -N + \frac{1}{2} & \dots & -\frac{1}{2} & \frac{1}{2} & \dots & N - \frac{1}{2} \end{array} \quad (10.22)$$

We recall from Chapter 3, Table 3.3, Property 8, that the time shift described in (10.22), that is, the left shift of $N - 1/2$ steps, causes the DFT, U_m^{DFT} , to be multiplied by the factor $e^{j[2\pi m(N-1/2)/2N]}$. The transform, V_m^{DFT} , of the shifted time series is given by

$$\begin{aligned} V_m^{\text{DFT}} &= U_m^{\text{DFT}} e^{j \frac{2\pi m(N-1/2)}{2N}} \\ &= \sum_{k=0}^{N-1} x_k e^{j \frac{2\pi m(k+1/2)}{2N}} + \sum_{i=0}^{N-1} x_i e^{-j \frac{2\pi m(i+1/2)}{2N}}; \quad m = 0, 1, \dots, 2N-1 \end{aligned} \quad (10.23)$$

The two sums in the last line are conjugates of each other, so their imaginary parts cancel. Therefore, by limiting m in (10.23) to its first N values, we can write an expression that contains all of the DCT components in (10.18):

$$\begin{aligned} V_m^{\text{DFT}} &= 2\text{Re} \left\{ \sum_{i=0}^{N-1} x_i e^{-j \frac{2\pi m(i+1/2)}{2N}} \right\} \\ &= 2 \sum_{n=0}^{N-1} x_n \cos \left(\frac{(2n+1)m\pi}{2N} \right) = \frac{2X_m^{\text{DCT}}}{c_m}; \quad m = 0, 1, \dots, N-1 \end{aligned} \quad (10.24)$$

Thus, we have a derivation of the DCT in terms of a related DFT. The components in (10.18) are scaled to produce a symmetric inverse, as we shall see. But first, we note that the upper sum in (10.24) is the DFT of x appended with N zeros and delayed (shifted right) by $1/2$ sample. There are two consequences of this. First, we see that the components of the DCT of a vector, x , are nearly the same as the components of the DFT of the vector $[x' \ x]$, and thus, we would expect the DCT and the DFT to produce similar spectra. The similarity is illustrated in Figure 10.11, where DFT and DCT amplitudes are plotted for comparison. The signal, in this case, is a portion of the speech waveform in Figure 8.17. Note that, although the DCT has twice as many components, the two amplitude spectra are similar in form.

Second, we have in the first line of (10.24) a way to compute the DCT using the FFT algorithm; that is, an equivalent expression for X , the DCT of x , is

$$\begin{aligned} y &= [x_0 \ x_1 \ \dots \ x_{N-1} \ 0 \ 0 \ \dots \ 0] \\ X_m &= c_m \text{Re} \left\{ \text{FFT} \{y\} e^{-j \frac{m\pi}{2N}} \right\}; \quad m = 0, 1, \dots, N-1 \end{aligned} \quad (10.25)$$

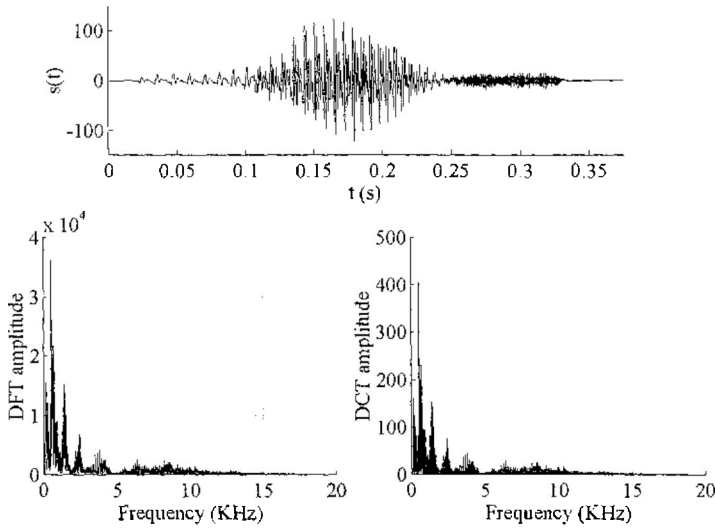


FIGURE 10.11

A segment of the speech signal, $s(t)$, in Figure 8.17 with DFT and DCT amplitudes, showing the similarity between the two transforms.

The zeros in (10.25) increase the FFT size from N to $2N$, and there are more efficient algorithms for computing the DCT,⁷ but the form in (10.25) is useful if only the FFT is available.

By the same line of reasoning, a formula equivalent to (10.25) for the inverse DCT in (10.19) is

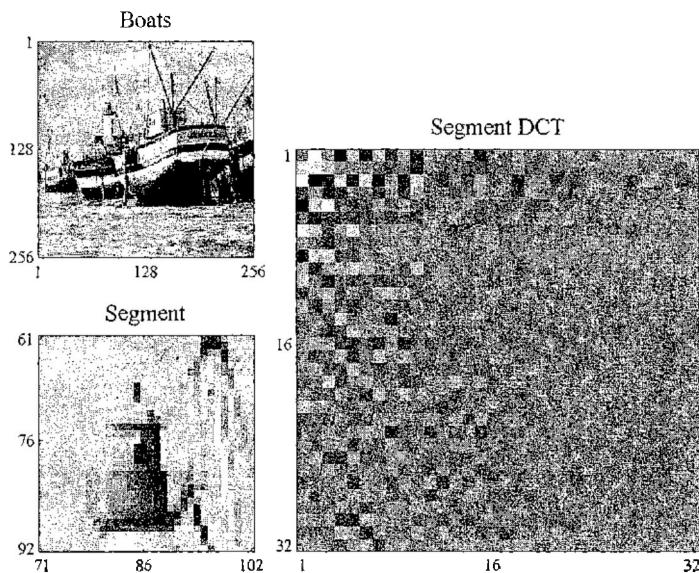
$$Y_m = c_m e^{j \frac{m\pi}{2N}} X_m^{\text{DCT}}; \quad m = 0, 1, \dots, N-1;$$

$$x_n = 2N \text{Re}\{ \text{IFFT}\{[Y_0 \ Y_1 \ \dots \ Y_{N-1} \ 0 \ 0 \ \dots \ 0]\}\}; \quad n = 0, 1, \dots, N-1$$

(10.26)

Two MATLAB functions, $y = \text{sp_dct}(x)$ and sp_idct , that compute the DCT and its inverse using (10.25) and (10.26) are included to demonstrate these methods. The derivation of (10.26) is similar to the derivation of (10.25) and is left for Exercise 26.

In image coding and spectral analysis, we use a *two-dimensional transform*. In general, the two-dimensional transform of an array consists of transforming the *columns* of the array one at a time, and then transforming the *rows* of the result. Thus, the two-dimensional transform is quite different from the one-dimensional transform. Recall that the DFT of an array, for example, is given as the DFT of each column. Thus, the two-dimensional DFT, which

**FIGURE 10.12**

Two-dimensional DCT of a segment of the “Boats” image. The segment includes the top of the tower in the upper left quadrant of the complete image.

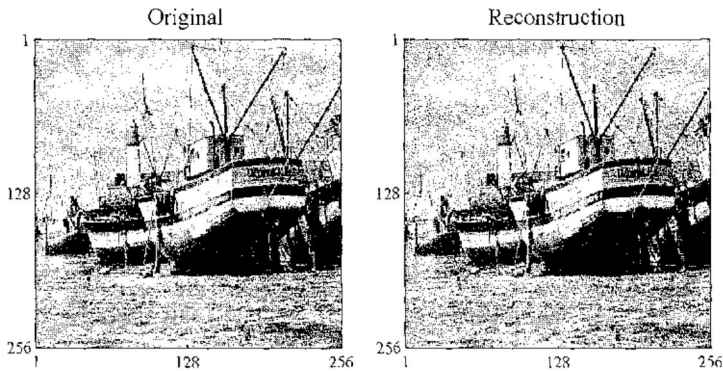
we call DFT2 , is produced as follows, using “.” to indicate the transpose:

$$(\text{Two-dimensional}) \text{DFT2}(x) = \text{DFT}\{\text{DFT}\{x\}.'\}.' \quad (10.27)$$

The MATLAB `fft2` function accomplishes this computation.

An illustration of the two-dimensional DCT in image coding is given in Figure 10.12. The two-dimensional DCT is computed as in (10.27) with the DCT in place of the DFT. In the illustration, we use the image known as “Boats,” which appears often in DSP literature. The complete image, a 256×256 array of 8-bit gray-scale pixels, is shown on the upper left. On the lower left, we extracted a 32×32 -pixel segment consisting of the top of the tower in the upper left quadrant of the complete image. The DCT of this segment is shown on the right in Figure 10.12. To plot the DCT segment, the DCT components were scaled to the image pixel range, that is, $[0, 255]$. In this representation of the DCT, *low-frequency* components appear at the upper left, *high-frequency* components are at lower right, and component *values* increase from black to white.

Compared with the original image, the rounded DFT and the rounded DCT in this case have lower entropies per element, as seen in the following data. (For DFT entropy, we used a comparable array consisting of separate real and imaginary parts with eight bits per element.) These entropies are

**FIGURE 10.13**

Original image and reconstruction after 4:1 compression using two-dimensional DCT quantization.

for the complete image, not the segment, in Figure 10.12:

Entropy of Boat's image:	7.19 bits/pixel	
Entropy of DFT components:	1.50 bits/component	(10.28)
Entropy of DCT components:	1.76 bits/component	

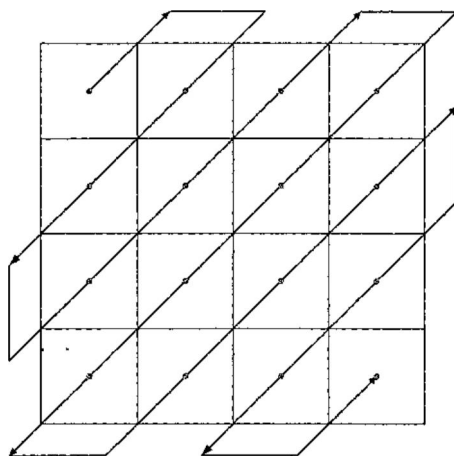
Thus, transform coding reduces the image entropy. Furthermore, as seen here, the choice of the DCT over the DFT is due not to entropy, but to the real component values in the DCT, which are more amenable to quantization as well as to other procedures that lead to lossy image compression.

To produce the entropies above, the mean value of the image was subtracted before transforming, the transforms were scaled to the range $[0, 255]$, and the transform components were rounded to the nearest integer. In image compression, the information needed to undo these steps must be included with the data.

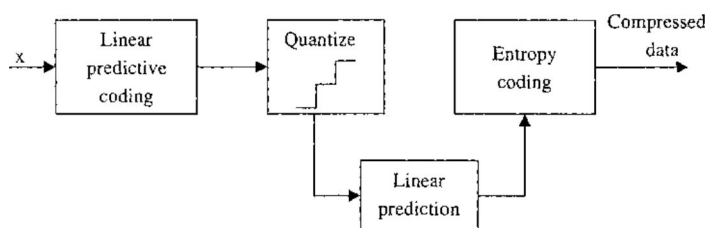
The rounding operation is a simple example of the quantizing step in Figure 10.10 which, in this case, results in a compression in terms of the entropy ratio of $7.19/1.76$, or about 4:1. With this degree of quantization, the quality of the reconstruction via the inverse DCT, which is shown in Figure 10.13, is good.

When the image transform is *scanned* to convert the array to a vector, as it is in some compression schemes, the *zig-zag scan* (Figure 10.14) is preferable, because it places components at similar frequencies near each other in the scanning sequence. A MATLAB function called $y = \text{image_scan}(x)$, for which x is any rectangular array, and y is the zig-zag scan vector, is included to accomplish this type of scan.

Because the DCT components are likely to be correlated along the scan line, linear predictive coding of these components prior to entropy coding, illustrated in Figure 10.15, may be advantageous. The result of adding this step, that is, zig-zag scanning followed by linear predictive coding, to the compression process for the Boats image reduces the DCT entropy slightly from the result in (10.28).

**FIGURE 10.14**

Zig-zag scanning of two-dimensional pixels or transform components.

**FIGURE 10.15**

A linear prediction stage added to the compression scheme in Figure 10.10.

10.6 Multirate Signal Decomposition and Subband Coding

Another method used to analyze and compress waveforms and images involves a process known as *multirate signal processing*, or *multirate signal decomposition*, combined with *subband coding*.^{19,22} These terms are more or less self-descriptive, as we shall see. Multirate processing is also related to signal transformation using *wavelets*. These are all broad subjects, and there are several good reference texts for the reader wishing to pursue them.¹⁹⁻²⁵ Our goal is to introduce the concepts and show how they apply to signal and image analysis and compression.

Because filtering is a familiar subject in DSP, it is easiest to describe these concepts in terms of filtering operations. The key process in this approach

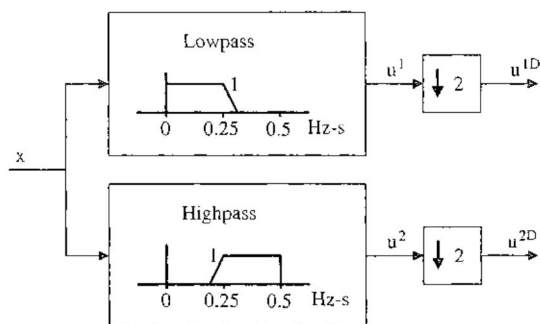


FIGURE 10.16
Single stage of a multirate signal processing system.

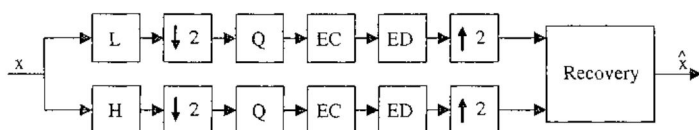


FIGURE 10.17
Encoding and recovery involving a wavelet transform with two components. Symbols Q , EC , and ED stand for quantization, entropy coding, and entropy decoding, respectively

is known as *decimation*, or *down-sampling*. A simple filtering and decimation process is illustrated in Figure 10.16. The principle is the same, as we shall see, for both waveforms and images, and, although only two filters and down-sampling by a factor of 2 are shown in Figure 10.16, the same analysis holds for any number (M) of filters and down-sampling by the factor M .

The down-sampling operations in Figure 10.16 are similar to down-sampling discussed in Chapter 5, Section 5.3. For our discussion, the signals in the lowpass channel are labeled as follows:

$$\begin{aligned} x &= [x_0 \ x_1 \ x_2 \ \dots] \\ u^1 &= [u_0^1 \ u_1^1 \ u_2^1 \ \dots]; \quad u^{1D} = [u_0^1 \ u_2^1 \ u_4^1 \ \dots] \end{aligned} \quad (10.29)$$

Thus, u^{1D} , the down-sampled or *decimated* version of u^1 , is a sequence consisting of every other sample of u^1 .

If the decimated sequences, u^{1D} and u^{2D} , are defined in this manner, then clearly these sequences together have the same number of samples as their parent vector, x . (We assume for this discussion that the length of x is even.) Furthermore, if, say, u^{2D} contains only a small amount of useful information compared with u^{1D} , then it is useful to be able to subject the two signals separately to quantization and entropy coding, that is, *subband coding*. An encoding and recovery concept is illustrated in Figure 10.17. L and H stand

for the lowpass and highpass filters in Figure 10.16, Q stands for quantizing, EC stands for entropy coding, and ED stands for entropy decoding. We assume EC - ED is a lossless process. We view this concept as encoding and recovery in a *multirate processor with two components* produced by the filters, L and H , each component the result of convolving x with the impulse response of the respective filter, as described in Chapter 4.

Notice that multirate processing differs from transforms like the DCT in that it involves digital filters that may be used to process a signal or image continuously rather than one vector at a time. This continuous type of processing in lossy compression eliminates *blocking effects* of the DCT in the recovered signal, which may be objectionable in waveform and image compression.

Multirate processing, with more components involves the application of the same type of filtering and down-sampling in cascade arrangements, as we will discuss shortly. The main subject in multirate theory, however, lies in the design of lowpass and highpass filters, which produce useful transformations and allow exact or at least acceptable recovery of the original signal.

The first question we might ask is whether the decimated sequences, u^{1D} and u^{2D} in (10.29), which contain the same number of samples as x , contain enough information to reconstruct x , even with ideal filters. Let us assume small (lossless) quantizing steps and lossless entropy coding, resulting in the version shown in Figure 10.18. Then the answer to our question lies in the relationship between the spectrum of a continuous signal and the spectrum of its sample vector. This relationship is given in Chapter 3, (3.42), which gives the spectrum of the samples as a superposition of shifted spectra of the continuous signal.

Suppose now that we have a waveform, $x(t)$, sampled with time step T , that is small enough to prevent aliasing. Let $X(\omega)$ represent the continuous DFT of $x(t)$ [as in (3.25), for example]. We use this representation here instead of the more correct notation, $X(e^{j\omega T})$, for the sake of brevity. Because the sampling theorem is satisfied, we note from (3.42) that $TX(\omega)$ in the range $|\omega| \leq \pi/T$ also represents the Fourier transform of $x(t)$. Suppose the Fourier transform of $x(t)$ has the form shown at the upper left in Figure 10.19. Note that the spectrum of $x(t)$ occupies the entire range from zero to half the sampling rate, so any attempt to down-sample x would result in an aliased reconstruction, as described in Chapter 3, Section 3.10, and particularly in (3.42).

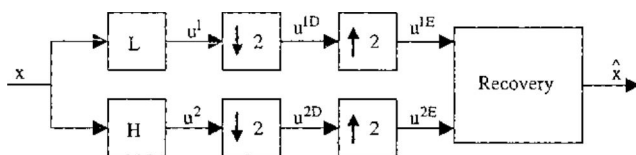


FIGURE 10.18

Equivalent of Figure 10.17 when the quantizing and entropy coding operations are lossless.

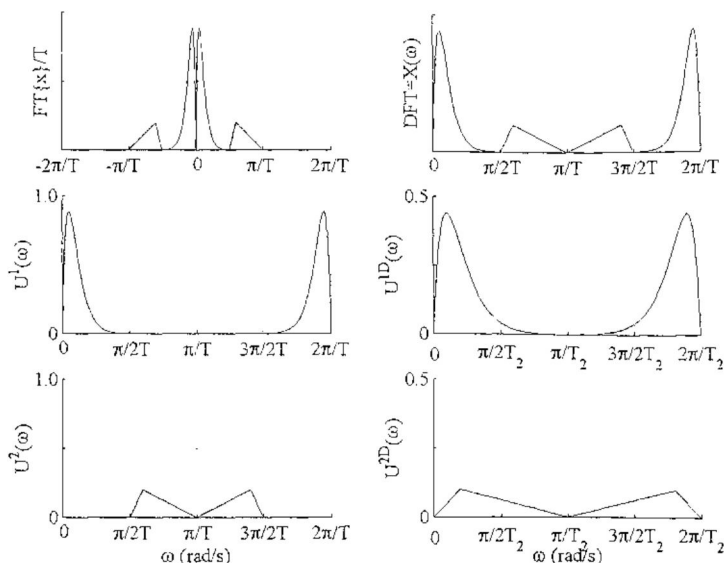


FIGURE 10.19

Upper left: continuous transform of $x(t)$ scaled by $1/T$. Upper right: DFT of sample vector x with time step T . Middle left: DFT of u^1 , which is x after lowpass filtering. Middle right: DFT of u^1 after down-sampling with $T_2 = 2T$. Lower plots: same as middle plots for u^2 , which is x after highpass filtering.

The DFT of a long vector, x , consisting of samples of $x(t)$ with time step T , is shown at the upper right in Figure 10.19. This is one cycle of the DFT, which we designate $X(\omega)$. The DFT consists of repetitions of $X(\omega)$ over all frequencies. We also note that these transforms are usually complex, and that we are using *real* transforms here for the sake of the illustration.

For this discussion, we assume the filters in Figure 10.18 are ideal, and produce u^1 and u^2 with DFTs $U^1(\omega)$ and $U^2(\omega)$ shown at the middle and lower left in Figure 10.19. That is, the filters extract the low- and high-frequency portions of $X(\omega)$ without overlap. Because the spectra of u^1 and u^2 are (ideally) band-limited, the DFTs of these down-sampled signals, $U^{1D}(\omega)$ and $U^{2D}(\omega)$, which are shown at the middle and lower right, are not aliased. Note that the time step after down-sampling is

$$T_2 = 2T \quad (10.30)$$

and therefore, the frequency range in the plots of $U^{1D}(\omega)$ and $U^{2D}(\omega)$ is half the range in the plots of $U^1(\omega)$ and $U^2(\omega)$. Also, the spectral values are only half as large for the decimated sequences, because these have only half as many samples. Thus, all the information in $X(\omega)$ (and thus in x) is contained

unambiguously in the down-sampled signals, u^{1D} and u^{2D} , and recovery of x is at least theoretically possible.

We now consider the question of recovery. As shown in Figure 10.18, recovery is preceded by *up-sampling*, or *expansion*. In general, expansion by a factor M consists of inserting $M - 1$ zeros between each pair of samples and is thus an application of Property 6 in Chapter 3, Table 3.3, which states that the DFT of the expanded sequence consists of M repetitions of the DFT of the sequence before expansion.

In the case of Figure 10.18 where the factor is $M = 2$, we have the following sequences:

$$\text{Expansion; } M = 2: \begin{cases} u^{1E} = \{u_0^{1D} & 0 & u_1^{1D} & 0 & u_2^{1D} \dots\} \\ u^{2E} = \{u_0^{2D} & 0 & u_1^{2D} & 0 & u_2^{2D} \dots\} \end{cases} \quad (10.31)$$

In accordance with Property 6, then, u^{1E} has a DFT, U^{1E} , consisting of two repetitions of U^{1D} , and likewise for u^{2E} . This result is illustrated in Figure 10.20. Again, because $T_2 = 2T$, the frequency range on the right in Figure 10.20 is twice that on the left. We can also see that an ideal lowpass filter would succeed in extracting the portion of U^{1E} from 0 to $\pi/2T$ rad/s, and an ideal highpass filter would succeed in extracting the portion of U^{2E} from $\pi/2T$ to π/T rad/s. Thus, the ideal recovery of x from its decimated components would be accomplished as in Figure 10.21, and we may view Figure 10.21 as a simple example of a transform (in the left half of the figure) and its inverse (in the right half).

Practical multirate processors, however, use real digital filters (both FIR and IIR) in place of the ideal filters (L and H) and actually *allow aliasing* to occur during the transformation and recovery. Aliasing causes signal spectra to be

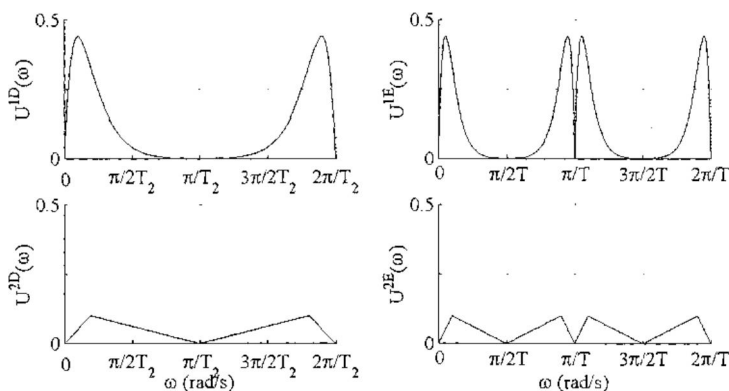


FIGURE 10.20

DFTs of the down-sampled sequences on the left; DFTs of the expanded sequences on the right. With $T_2 = 2T$, expansion consists of inserting a zero between each pair of samples in the down-sampled sequences. Frequency ranges on the right are twice those on the left.

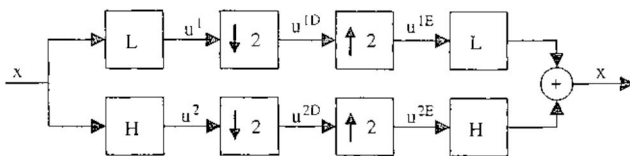


FIGURE 10.21

Ideal decomposition and recovery of a signal using decimation with $M = 2$.

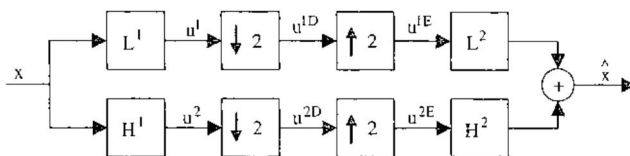


FIGURE 10.22

Decomposition and recovery of a signal using real filters that allow aliasing.

modified as described in Chapter 3, Sections 3.10 and 3.11, and illustrated, for example, in Figure 3.16. In the present case, aliasing would cause the spectral components in Figure 10.20 to overlap and add together in the vicinity of $\omega = \pi/T_2$ in the left-hand plots, and in the vicinity of $\omega = \pi/2T$ and $\omega = 3\pi/2T$ in the right-hand plots, as described by (3.42) in Chapter 3.

With aliasing present, we now consider the configuration in Figure 10.22, in which the input and recovery filters (L^1 and L^2 , or H^1 and H^2) are not necessarily the same. In this configuration, we have the following DFT relationships for the *forward transform*:

$$\begin{aligned} U^1(\omega) &= X(\omega)L^1(\omega); & U^{1D}(\omega) &= \frac{1}{2}\left(U^1(\omega) + U^1\left(\omega - \frac{\pi}{T}\right)\right) \\ U^2(\omega) &= X(\omega)H^1(\omega); & U^{2D}(\omega) &= \frac{1}{2}\left(U^2(\omega) + U^2\left(\omega - \frac{\pi}{T}\right)\right) \end{aligned} \quad (10.32)$$

In each line, the first expression describes the input filtering operation. The second expression is due to aliasing in the vicinity of $\omega = \pi/T_2$ and amounts to an application of (3.42) with two terms (converted to DFTs), one for $m = 0$ and the second for $m = 1$.

For the *inverse transform*, that is, the second half of Figure 10.22, the following relationships are applicable in the range $|\omega| \leq \pi/T$:

$$\left. \begin{aligned} U^{1E}(\omega) &= U^{1D}(\omega); \\ U^{2E}(\omega) &= U^{2D}(\omega); \end{aligned} \right\} \hat{X}(\omega) = L^2(\omega)U^{1E}(\omega) + H^2(\omega)U^{2E}(\omega) \quad (10.33)$$

The first two equations here are illustrated in Figure 10.20, and the third is evident from Figure 10.22.

Before proceeding to see whether $\hat{X}(\omega)$ can be made equal to $X(\omega)$ in the presence of aliasing, we express the DFTs in (10.32) and (10.33) using z -transform notation, that is, with

$$z = e^{j\omega T} \quad (10.34)$$

as in Chapter 4, (4.5). Then each DFT of the form $X(\omega)$, or more precisely, $X(e^{j\omega T})$, becomes $X(z)$, and each DFT of the form $U(\omega - \pi/T)$ becomes $U(-z)$ via (10.34). If we make these substitutions in (10.33) and (10.34) and combine terms, we obtain the following result:

$$\begin{aligned} 2\hat{X}(z) &= L^2(z)[X(z)L^1(z) + X(-z)L^1(-z)] + H^2(z)[X(z)H^1(z) + X(-z)H^1(-z)] \\ &= X(z)[L^1(z)L^2(z) + H^1(z)H^2(z)] + X(-z)[L^1(-z)L^2(z) + H^1(-z)H^2(z)] \end{aligned} \quad (10.35)$$

With $\hat{X}(z)$ expressed in this form, it is now easy to proceed.

To prevent aliasing in the output, we must not allow $X(-z)$ to be part of its construction. Thus, in the second line of (10.35),

$$\text{To prevent aliasing: } L^1(-z)L^2(z) + H^1(-z)H^2(z) = 0 \quad (10.36)$$

With this constraint on the filters, the second term on the second line in (10.35) vanishes, and we can see the *additional* requirement for exact recovery of the input signal:

$$\text{For exact recovery: } L^1(z)L^2(z) + H^1(z)H^2(z) = 2 \quad (10.37)$$

We consider now the choice of the input filters, L^1 and H^1 . These are often chosen by first designing $L^1(z)$ as a linear-phase FIR filter, and then specifying

$$H^1(z) = L^1(-z); \quad \text{that is, } H^1(\omega) = L^1(\omega - \pi/T) \quad (10.38)$$

With this specification, we create (in terms of the DFT) $H^1(\omega)$ as the *mirror image* of $L^1(\omega)$, as illustrated in Figure 10.23. Again, note that the plots

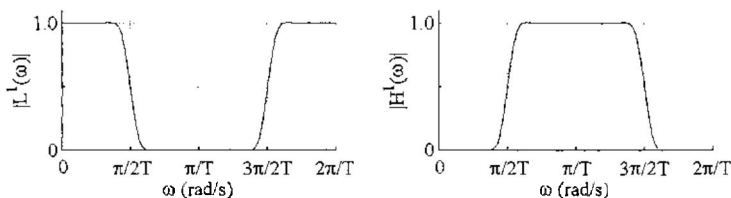
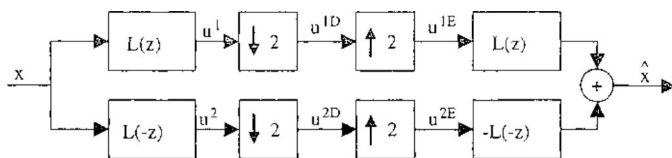


FIGURE 10.23

Amplitude gains of lowpass and highpass FIR mirror-image filters, with $H^1(\omega)$ equal to $L^1(\omega - \pi/T)$.

**FIGURE 10.24**

Signal decomposition and exact recovery with quadrature mirror filters. $L(z)$ is any lowpass filter with cutoff at $\omega_c = \pi/4T$, and the other filters are determined from $L(z)$.

show DFT amplitudes over the range $0 \leq \omega \leq 2\pi/T$, rather than the usual range, $0 \leq \omega \leq \pi/T$, in order to emphasize the frequency shift in (10.38). Thus, with (10.38), we assure that if $L^1(z)$ is a good lowpass filter, then $H^1(z)$ is a good highpass filter.

If we now impose the constraint in (10.38), we can eliminate $H^1(z)$ in (10.36) and (10.37), with the following result:

$$\begin{aligned} \text{Prevent aliasing: } L^2(z)L^1(-z) + H^2(z)L^1(z) &= 0 \\ \text{Exact recovery: } L^2(z)L^1(z) + H^2(z)L^1(-z) &= 0 \end{aligned} \quad (10.39)$$

With these two constraints, we have now specified the recovery filters in terms of the single filter, $L^1(z)$, that is,

$$L^2(z) = L^1(z); \quad H^2(z) = -L^1(-z) = -H^1(z) \quad (10.40)$$

The result is shown in Figure 10.24, where there is now only a single choice of a lowpass filter, $L(z)$. Because the input filter gains are mirror images with respect to the quadrature frequency $\pi/4T$, the filters in this system are known as *quadrature mirror filters*.

Figure 10.24 contains the basic elements of this introductory discussion of multirate processing. The first half of the figure represents a *signal decomposition*, and the second half represents an *expansion*, or *recovery*, of the signal. There are other considerations in the design and use of filters for multirate processing, the selection of the factor M , etc., including the use of adaptive filters in multirate systems, which are beyond our scope here. The reference by P.P. Vaidyanathan²⁰ is particularly recommended for further study.

We can, however, make some observations about Figure 10.24 and its use in signal analysis and compression, as discussed, for example, with Figure 10.17. First, each filter in Figure 10.24 introduces a phase shift that we have ignored by plotting real spectra in Figures 10.19 and 10.20. If the filters are linear-phase FIR filters such as those described in Chapter 5, the phase shift from x to $\hat{x}$ in Figure 10.24 amounts to a signal delay of $N - 1$ samples, where N is the number of weights in each filter. Thus, the entire

process can be made causal without causing distortion. Exercises 22 and 23 illustrate this point.

Second, as we noted before, any transform that processes a waveform vector can be made two-dimensional, as discussed previously in connection with the FFT and the DCT. Let $TR(x)$ represent any transformation of the signal vector x . When x is an array, $TR(x)$ is defined to operate on the *columns* of x . Then the two-dimensional transform, $TR2$, is defined always as follows:

$$TR2\{x\} = TR\{TR\{x\}.'\}.'$$
(10.41)

In the case of the multirate processing, the “transform” may be formed by ordering the output sequences from low to high frequency to form the output vector in each column of the transformed array.

Third, we also observe that the configuration in Figure 10.24 may be expanded in different ways to form a complete transform. Two of the most common ways are illustrated in Figures 10.25 and 10.26. In Figure 10.25, we see the first two stages of a decomposition with *equal bands*. Each successive stage decomposes its input vectors into twice as many output vectors, each output vector having half the number of samples in one of the input vectors. Thus, the frequency domain of x is partitioned into twice as many equal

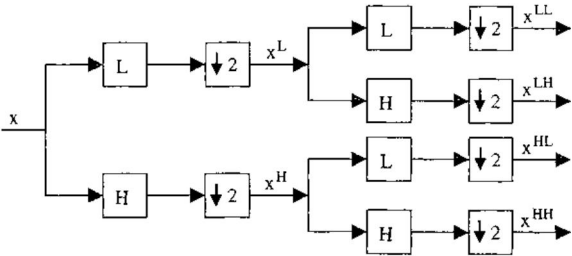


FIGURE 10.25
Decomposition with equal bands.

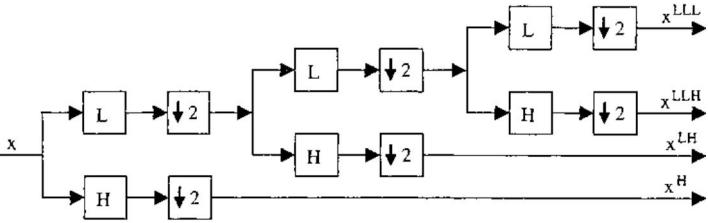


FIGURE 10.26
Decomposition with octave bands.

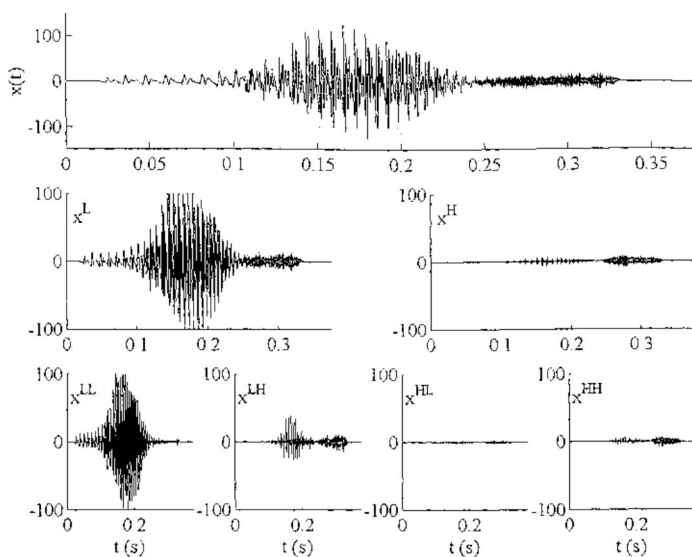


FIGURE 10.27

Decomposition of a short speech segment into equal bands in accordance with Figure 10.25.

parts with each stage, and a transform with N stages has 2^N components, that is, decimated waveform vectors. Note the *superscript* labeling of the down-sampled vectors in Figure 10.25 (and 10.26 as well). These indicate the relative frequency bands, and the *number* of superscripts tells us how many times the length of the input has been reduced by a factor of two due to down-sampling.

Decomposition with *octave bands* is illustrated in Figure 10.26. In this case, only the low-frequency component is decomposed at each stage. The applicability of this approach may be seen by examining the speech spectra in Figure 10.11 and the image spectrum in Figure 10.12. In general, the spectra of naturally occurring waveforms and images tend to be concentrated at low frequencies, and the amplitude spectrum tends to change at a faster rate at lower frequencies, thus justifying the uneven bandwidths in the transform. There is an advantage with the octave bands, because decomposition with N stages produces $N + 1$ components instead of the 2^N components produced in the equal-band transform.

Using the speech signal in Figure 10.11, the equal- and octave-band decompositions are illustrated in Figure 10.27, which shows all the signals in the system, and Figure 10.28, which shows just the input and output signals. In the octave-band decomposition, we can observe that the signal energy is distributed more uniformly in the three high-frequency bands, and note that more stages in the system would decompose x^{LLL} without changing the other three signals.

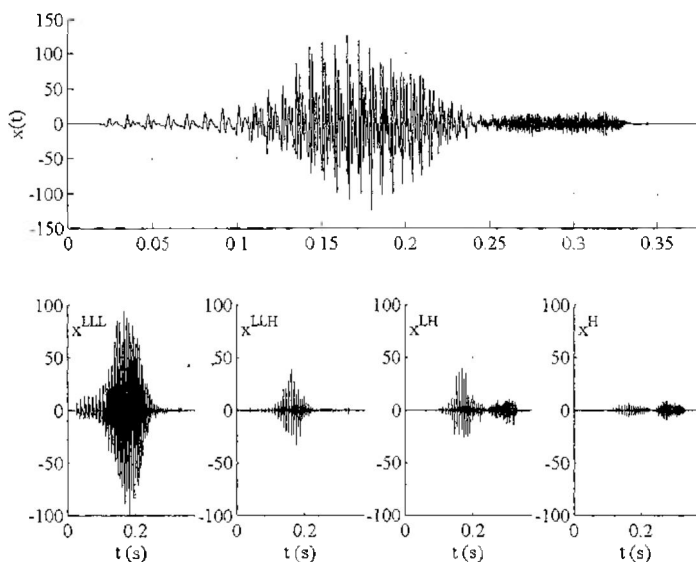


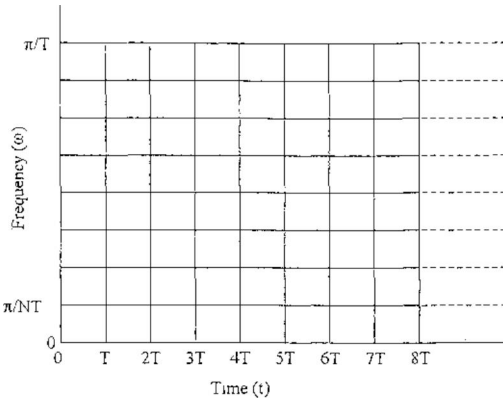
FIGURE 10.28

Decomposition of a short speech segment into octave bands in accordance with Figure 10.26.

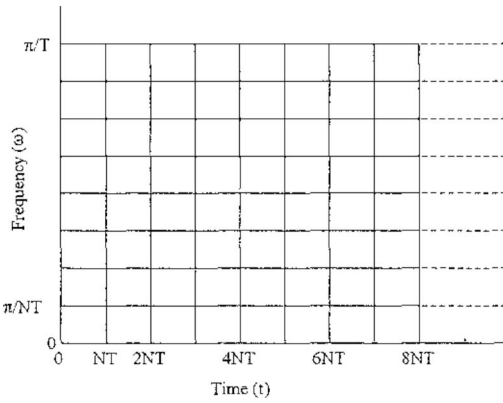
10.7 Time-Frequency Analysis and Wavelet Transforms

In this final section, we introduce the general concept known as *time-frequency analysis* as a method for analyzing and transforming signals. Time-frequency analysis is closely related to the transforms we discussed, that is, the DFT and the DCT, and also to multirate processing and filtering. To begin, let us reconsider the *spectrogram*, which was introduced in Chapter 6. In the illustration in Figure 6.20, we had an example of partitioning the time-frequency plane as might be represented by Figure 10.29. The plane is divided into segments such that the time increment is the time step, T , and the frequency increment is π/NT , where N is the number of bandpass filters, that is, teeth in the comb filter. The height of each rectangle in Figure 10.29, which is π/NT rad/s, then represents the *frequency resolution*, that is, the pass-band of the corresponding tooth in the comb filter. The width represents the *time resolution*, that is, the single step from kT to $(k+1)T$ in the output of the filter.

Now instead of the comb filter in Chapter 6, suppose we define a *short-time discrete transform* using a scheme similar to that used for spectral analysis in Chapter 7. That is, we use a sliding window, w_{k-nN} , of length N and take (say) the DFT or the DCT at intervals of NT seconds (s) in time, resulting in a different set of N transform components every N time steps and partitioning the time-frequency plane as illustrated in Figure 10.30. (In this case, the transform

**FIGURE 10.29**

Representation of how the spectrogram resolves time and frequency. Each waveform in the spectrogram is within a bandwidth of π/NT rad/s, and the points on each waveform are spaced T s apart.

**FIGURE 10.30**

A short-time transform, such as the short-time DCT, partitions the time-frequency plane into equal parts. The frequency resolution is π/NT rad/s, and the time resolution is NT s, where N is the length of the time-domain window.

components are saved rather than averaged as they were in Chapter 7.) The short-time DCT may be described by modifying (10.18) as follows:

$$X_m^{\text{DCT}} = c_m \sum_{k=0}^{\infty} x_k w_{k-nN} \cos\left(\frac{(2k+1)m\pi}{2N}\right); \quad m = 0, 1, \dots, N-1; \quad n = 0, 1, \dots, \infty \quad (10.42)$$

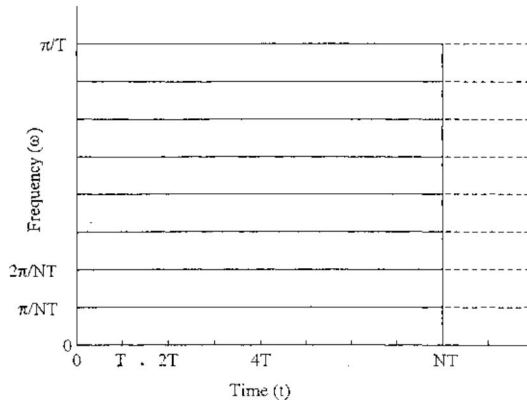


FIGURE 10.31

Time-frequency resolution for the equal-band multirate filter with three stages and $N = 8$ output signals. Frequency resolution is π/NT , and with down-sampling, the time resolution becomes NT s.

In this description, w_{k-nN} is the window that covers the time domain from nNT to $(n + 1)NT$ s and selects the sequence in the range $[x_{nNT} \dots x_{(n+1)NT}]$. We now have a frequency resolution equal to π/NT rad/s and a time resolution equal to NT s, as shown in Figure 10.30. Note that the implied constant area (π) of each rectangle illustrates the *uncertainty principle*. If we increase N in order to increase frequency resolution, we must accept a proportionate decrease in time resolution.

In the case of the short-time discrete transform, we have something quite similar to the equal-band multirate filter described previously. Down-sampling by the factor N , as illustrated, for example, for $N = 4$ in Figure 10.25, increases the time step from T to NT , as we have seen, and produces signals with frequencies at multiples of π/NT rad/s. Thus, the time-frequency resolution for equal-band multirate processing is also depicted in Figure 10.30 and is the same as that for the short-time DCT. To emphasize the time resolution produced by down-sampling, we use Figure 10.31, which illustrates the same resolution in frequency and time with three stages, that is $N = 8$.

Short-time transforms and equal-band coding do not, however, have the advantage of *octave-band* coding with its nonuniform frequency resolution. This is an important advantage to have with the encoding of data from natural sources, because, as we mentioned previously, the spectra of these data generally tend to decrease in proportion to frequency, that is, as $1/\omega$. In the example used in Figure 10.26, octave-band coding partitions the time-frequency plane as shown in Figure 10.32. Note that the area of each rectangle (time-frequency resolution) is constant (π).

Wavelet transforms^{20,22–26} may be viewed as a generalization of the octave-band coding concept. Suppose we view the three-stage system in Figure 10.26

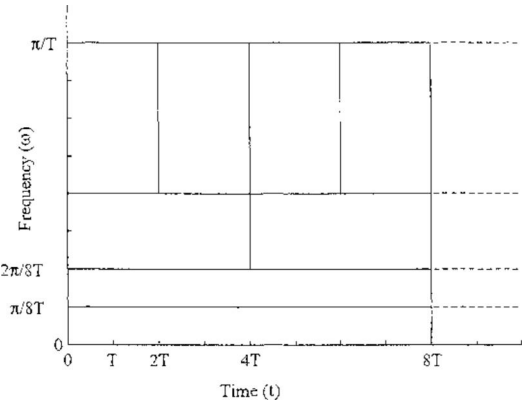


FIGURE 10.32 Time-frequency resolution for the octave-band multirate filter with three stages and four output signals. Frequency resolution decreases with frequency, time resolution increases with frequency, and the product of the two remains constant and equal to π .

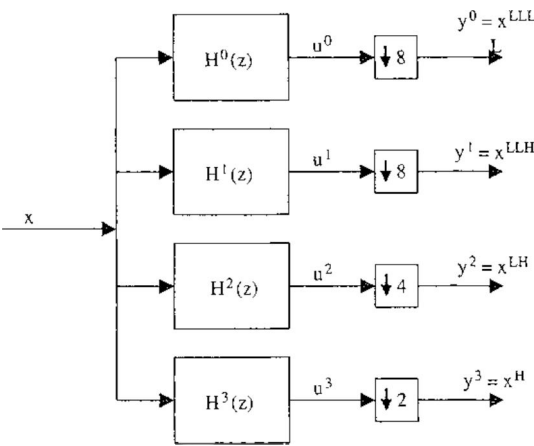


FIGURE 10.33 Filter structure for the wavelet transform, which is the same as the octave-band structure in Figure 10.26 with filtering and down-sampling combined into single operations. The wavelet transform components are y_1 through y_4 .

as a single filter bank with varying bandwidths, as in Figure 10.33. Each transfer function, $H^m(z)$, is the overall transfer function of the corresponding channel in Figure 10.26, with down-sampling occurring all at once instead of one stage at a time. For convenience, we assume the signal is unbounded and (as before) the filters are noncausal, with impulse responses centered around

zero and also unbounded, so in the time domain, we may express each filter output as follows:

$$U^m(z) = X(z)H^m(z); \quad u_k^m = \sum_{n=-\infty}^{\infty} x_n h_{k-n}^m; \quad m = 0, 1, 2, 3; \quad -\infty < k < \infty \quad (10.43)$$

Each output, y^m , is down-sampled according to its position in Figure 10.33, and so we have

$$y_k^m = \sum_{n=-\infty}^{\infty} x_n h_{ik-n}^m; \quad \begin{cases} m = 0, 1, 2, 3 \\ i = 8, 8, 4, 2 \\ -\infty < k < \infty \end{cases} \quad (10.44)$$

It is easy to see how to extend this and add stages to Figure 26; for example, $m = [0 \ 1 \ 2 \ 3 \ 4]$ and $i = [16 \ 16 \ 8 \ 4 \ 2]$, and so on. The decimated signals, y_k^m are called *wavelet coefficients*, and together they comprise the *wavelet transform* of x . The *wavelets*, h_n^m , may be derived in different ways, and their derivation is the subject of texts on wavelet transforms, several of which are listed in the references. At the time of this writing, MATLAB also offers a special toolbox of wavelet transforms.

10.8 Exercises

General Instructions: A few of the exercises in this section involve processing a signal from the *dsp data* folder, which is part of the software with this text. When this is the case, you may use the MATLAB file *display_data* to display all of the signals in a single figure, and then use the expressions in *display_data* to read a specific file. In doing so, you may need to modify the path to the folder. Executing *display_data* should result in a color version of the two signals and the image in Figure 10.34.

1. The gray-scale image in Figure 10.13 has 256 pixels in each of 256 rows. Each pixel has a range of 256 levels of gray, ranging from black (0) to white (255).
 - a. How many different images are possible?
 - b. If all possible images are equally likely, how much information is contained in a single image?
2. You receive the following binary message from a source which sends one or zero with equal probability:

$$x = [0 \ 1 \ 0 \ 0 \ 0 \ 0 \ 1 \ 1 \ 0 \ 1 \ 1 \ 0 \ 1 \ 0 \ 0 \ 0 \ 0 \ 1 \ 1 \ 0 \ 1 \ 0 \ 0 \ 1 \ 0 \ 1 \ 1 \ 1 \ 0 \ 0 \ 0 \ 0]$$

- a. What is the entropy of the message, based on the source probabilities?

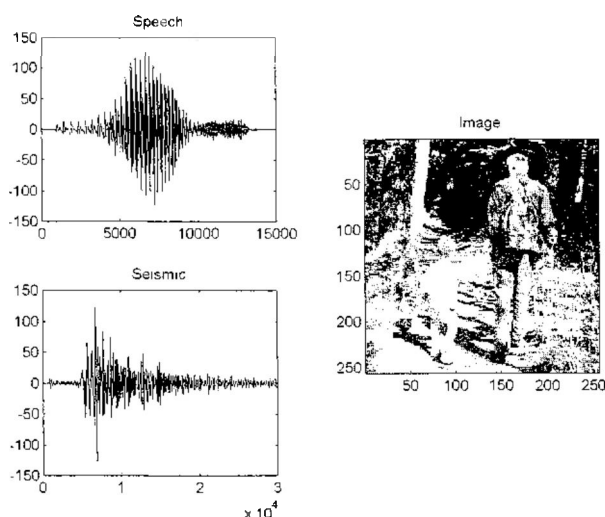


FIGURE 10.34

Speech waveform, seismic waveform, and image used in several of the exercises.

- b. What is the entropy measure that would be used for encoding x , based on its content?
 - c. The message actually consists of consecutive 8-bit ASCII symbols for the word "Chip." Compute the information received in these four symbols, assuming all ASCII symbols are equally likely.
3. Suppose we wish to encode the following message using a Huffman or arithmetic code:

Hatred stirs up strife, but love covers all offenses.

 - a. How many symbols are in the message, including spaces and punctuation?
 - b. How many *different* symbols are in the message, including spaces and punctuation?
 - c. Write all the different symbols in a row vector, s . Underneath s , write a vector f containing the symbol frequencies expressed as fractions. Assume that s and f are to be included with the encoded information.
 - d. Show how to compute the average entropy of the symbols in the message, and compute it.
4. Suppose the sample vector, x , plotted with the circular ($\circ$) symbol in Figure 10.1, is to be entropy-encoded using its own statistics. What is the smallest number of bits you could have in the encoded version, not counting the frequency table and other similar extra information?

5. What is the average entropy in bits per symbol of the two waveforms in Figure 10.1, taken together?
6. Given the sample vector, x , plotted with the circular ($\circ$) symbol in Figure 10.1, complete the following:
 - a. Describe the symbol set.
 - b. Develop a frequency table in terms of $[Nf_x(n)]$.
 - c. Develop a Huffman code tree following the procedure in (10.9), and draw the tree.
 - d. Using the code tree, tabulate the set of symbol codes and code lengths.
 - e. Encode the entire message, and express the result as a binary string.
 - f. Compute the total entropy of x , and compare the result with the length of (e).
7. (See the note above Exercise 1.) Quantize the speech signal in Figure 10.34 to integers in the range $[0,7]$, plot the result, and do parts (a) through (f) in the previous exercise.
8. (See the note above Exercise 1.) For the speech signal in Figure 10.34, complete the following:
 - a. Compute the minimum number of bits required for entropy coding.
 - b. Use the `h_codes` function to produce the complete set of binary symbol codes. Print the codes in binary. Find the shortest code, and verify its prefix property.
 - c. Compute the length of the encoded signal and compare with (a). (You do not need to construct the encoded signal.)
9. Given the sample vector, x , plotted with the circular ($\circ$) symbol in Figure 10.1, complete the following:
 - a. Describe the symbol set.
 - b. Develop and print a frequency table.
 - c. Use function `a_encode` to encode the entire message. Express the result as a binary string.
 - d. Compute the total entropy of x and compare the result with the length of (c).
10. Given the message vector in Exercise 3, assume the symbol set consists only of the ASCII symbols actually used in the message, and do parts (a) through (d) in the previous exercise.
11. (See the note above Exercise 1.) Let $x(1:30000)$ represent the seismic signal in Figure 10.34.
 - a. Compute the minimum number of bits required for entropy coding of $x(9000:9999)$.

- b. Use the `a_encode` function to encode $x(9000:9999)$. Print the first 32 bits of the output.
- c. Compute the length of the encoded signal and compare with (a).
- 12. (See the note above Exercise 1.) Let x represent the seismic signal in Figure 10.34.
 - a. Filter x using a least-squares, one-step predictor with six weights. Plot x and prediction error, e , in separate subplots. Do not include startup values e_0 – e_5 .
 - b. Encode e with e_0 – e_5 excluded. Compare the length of the Huffman-coded version with the entropy of x and the entropy of e .
- 13. (See the note above Exercise 1.) Let x represent the seismic signal in Figure 10.34.
 - a. Compute the DFT vector, X . Append the imaginary part of X to the real part. Scale the resulting real vector (Y) to the range $[0,255]$, and plot the result.
 - b. Using Huffman coding, compare the length of the encoded version of Y with the entropy of Y and the entropy of X .
 - c. Do part (b) using arithmetic coding. Comment on the results of (b) and (c).

Exercises 14 through 19 are specified in the table below. In each case, a waveform is processed in the manner described. The note above Exercise 1 explains how to obtain and plot the signals. In each exercise, do the following:

- a. Process the signal before coding as described, except in Exercises 14 and 15.
- b. Plot the signals (and processed signals, except in Exercises 14 and 15).
- c. Shift the values so the minimum is zero and the range is $[0,255]$. Construct a Huffman code list consisting of [symbol, code length] for each symbol, and print the list.
- d. Without actually creating the encoded message, compute the message length and the resulting compression ratio assuming eight bits per symbol in the original signal, and compare it with the results of other exercises you have worked in the same column of the table below.

	Speech Signal	Seismic Signal
Huffman coding only	Ex. 14	Ex. 15
Predictive coding quantized to integers; six weights Huffman coding	Ex. 16	Ex. 17
DCT of (signal - mean value), quantized to eight bits Huffman coding	Ex. 18	Ex. 19

20. Process the speech signal as in Exercise 18, but in place of the DCT, use a four-stage octave-band encoder with FIR filters having seven weights each and using the Hamming window.
 - a. Plot the concatenated encoder output, from low to high frequency.
 - b. Translate the output to integers in the range [0,255]. Find and print a set of Huffman codes.
 - c. Compute the length of the encoded message and the compression ratio.
21. (See the note above Exercise 1.) For the color version of the image in Figure 10.34, note that the array dimensions are [256,256,3]; that is, there are three arrays containing the red, green, and blue pixels of the image. Compute the two-dimensional DCT of each of the three arrays and combine these into the array X with dimensions [256,256,3]. Delete the d. c. component, that is, $X(1,1,:)$. Create a color figure with two images, each about 3.5" on a side. Put the original color image on the left, and on the right, display the segment $X(1:10,1:10,:)$ as a color image. Comment on how the two-dimensional DCT components change with the spacial frequency.
22. Refer to the diagram in Figure 10.24, which illustrates a quadrature-mirror filter (qmf) stage with recovery. Make a figure with the following eight subplots (4 by 2), in order:
 1. Chirping sinusoidal signal, x_n vs. n , with 128 samples given by

$$x = \sin(2 * \pi * [0:127] * \text{linspace}(.1, .4, 128) / 4) .$$

2. Amplitude gain of $L(z)$, a lowpass, linear-phase FIR filter designed with $N = 13$ weights and the Hamming window, and also the amplitude gain of the mirror filter, $L(-z)$.
 3. Signal u_n^{1D} vs. n .
 4. Signal u_n^{2D} vs. n .
 5. Signal u_n^{1E} vs. n .
 6. Signal u_n^{2E} vs. n .
 7. Reconstructed signal $\hat{x}_n$ vs. n .
 8. Signals x_n and $\hat{x}_{n-N}$ together vs. n . These two signals should be nearly the same. Explain why the shift of N samples is necessary to align the two signals.
23. Refer to the three-stage, octave-band filter in Figure 10.26. In this exercise, we will use a similar filter with $M = 4$ stages to partition the speech signal, x , described in the note above.
 - a. First, truncate x so its length, K , is the largest possible multiple of 2^M . What is the revised value of K ?

- b. Filter x through four stages with each stage using $N = 21$ weights and the Hamming window, producing eight signals similar to the four signals in Figure 10.27. Concatenate the eight signals into a single vector, e , with K samples.
 - c. Process e through the mirror image of the octave-band filter, and sum the result to produce the reconstructed signal, $\hat{x}$.
24. In Section 10.3 we saw how predictive coding could be applied to reduce the entropy of a signal prior to entropy coding, following the scheme in Figure 10.4. Apply this scheme to the waveform in Figure 10.8.
 - a. Apply the one-step predictor used in Figure 10.3, and make a plot similar to Figure 10.3 of the signal and the prediction error.
 - b. Compute the entropy of the original signal.
 - c. Compute and compare the entropy of the prediction error, e , excluding e_0 .
25. Prove by substitution into (10.18) that the inverse DCT formula in (10.19) is correct.
26. By using the inverse DFT formula, (3.23), prove that the fast version of the inverse DCT in (10.26) is identically the same as the original formula in (10.19).

References

1. Shannon, C.E., A Mathematical Theory of Communication, *Bell System Tech J.*, 27, 379 and 623, 1948.
2. Ingels, P.M., *Information and Coding Theory*, Intext, Scranton, PA, 1971.
3. Hamming, R.W., *Coding and Information Theory*, Prentice Hall, Englewood Cliffs, NJ, 1980.
4. Held, G. and Marshall, T., *Data Compression: Techniques and Applications, Hardware and Software Considerations*, John Wiley & Sons, New York, 1991.
5. Golomb, S.W., Peile, R.E., and Schotz, R.A., *Basic Concepts in Information Theory and Coding*, Plenum Press, New York, 1994.
6. Sayood, K., *Introduction to Data Compression*, Morgan Kaufman Publishers, Inc., 1996.
7. Rao, K.R. and Hwang, J.J., *Techniques and Standards for Image, Video, and Audio Coding*, Prentice Hall, Upper Saddle River, NJ, 1996, chap. 5.
8. Ziv, J. and Lempel, A., A Universal Algorithm for Sequential Data Compression, *IEEE Trans. on Information Theory* (23)3, p 337, May 1977.
9. Nelson, M.R., LZW Data Compression, *Dr. Dobbs Journal*, 29, 86, Oct. 1989.
10. Regan, S.M., LZW Revisited, *Dr. Dobbs Journal*, 126, 167, June 1990.
11. Gersho, A. and Gray, R.M., *Vector Quantization and Signal Compression*, Kluwer Academic Publishers, Norwell, MA, 1992.

12. Huffman, D.A., A method for the construction of minimum-redundancy codes, *Proc. IRE* 40, 1098, 1952.
13. Rissanen, J. and Langdon, G.G., Jr., Arithmetic coding, *IBM J. of Research and Dev.*, 23, 149, Mar. 1979.
14. Witten, I.H., Neal, R.M., and Cleary, J.G., Arithmetic coding for data compression, *Comm. ACM*, 30, 6, 520, June 1987.
15. Stearns, S.D., Arithmetic coding in lossless waveform compression, *IEEE Trans. on Signal Processing*, 43, 8, 1874, Aug. 1995.
16. Jayant, N.S. and Noll, P., *Digital Coding of Waveforms—Principles and Applications to Speech and Video*, Prentice Hall, Englewood Cliffs, NJ, 1984.
17. Ahmed, N. and Rao, K.R., *Orthogonal Transforms for Digital Signal Processing*, Springer-Verlag, New York, 1975.
18. Rao, K.R. and Yip, P., *Discrete Cosine Transform: Algorithms, Advantages, Applications*, Academic Press, Boston, MA, 1990.
19. Crochiere, R.E. and Rabiner, L.R., *Multirate Digital Signal Processing*, Prentice Hall, Englewood Cliffs, NJ, 1983.
20. Vaidyanathan, P.P., *Multirate Systems and Filter Banks*, Prentice Hall, Englewood Cliffs, NJ, 1993.
21. Shynk, J., Frequency-domain and multirate adaptive filtering, *IEEE SP Magazine*, 14, Jan. 1992.
22. Vetterli, M. and Kovacevic, J., *Wavelets and Subband Coding*, Prentice Hall, Englewood Cliffs, NJ, 1995.
23. Burrus, C.S., Gopinath, R.A., and Guo, H., *Introduction to Wavelets and Wavelet Transforms*, Prentice Hall, Upper Saddle River, NJ, 1998.
24. Boggess, A. and Narcowich, F.J., *A First Course in Wavelets with Fourier Analysis*, Prentice Hall, Upper Saddle River, NJ, 2001.
25. Strang, G. and Nguyen, T., *Wavelets and Filter Banks*, Wellesley-Cambridge Press, Wellesley, MA, 1996.
26. Saha, S., Image compression—from DCT to wavelets: A review, *ACM Crossroads* (ACM electronic publication), www.acm.org/crossroads, Feb. 2002.

Index

A

Adaptation

- geometry of, 241, 246
- as process, 241–242
- speed of, 263–264

Adaptive algorithm, 242

Adaptive arithmetic coding, 290

Adaptive array, 267–268

Adaptive beamforming, 268

Adaptive filter, 242, 267

Adaptive linear combiner, 268, 270

Adaptive predictor, least-mean-square, 258

Adaptive signal compression, 256–258

- gradient in, 244, 252, 264

Adaptive signal processing, 241–275

algorithms for

- direct descent, 258–263
- least-mean-square, 241, 249, 254–258
- one-step, 245
- other, 267–268
- random search, 243
- recursive least squares, 258–263
- steepest-descent, 249–254, 258

applications of, 73, 199

convergence in

- under ideal conditions, 246–249, 252

- learning curve for weights, 247–248, 255

- parameter for, 245–246

- as performance measure, 263–264, 266–267

- recursive least squares, 261–263

- steepest-descent algorithm, 249, 251–254, 258

- time constant for forgetting correlation estimate, 260

- time constant for LMS, 247–248

- time constant for MSE, 247–248

- time constant for weights, 246–247

- time constant under ideal conditions, 252

- weight factors, 246–247, 251–252, 255, 258, 263

- eigenvalues in, 250–253, 264, 266

- exercises for, 268–273

- finite impulse response filters in, 243–244, 267–268

- introduction to, 241–242

- mean-squared error in

- algorithm for, 241–242

- gradient of, 244

- least-mean-square convergence with, 255–257

- as performance measure, 263–267

- performance surface of, 242–244

- search algorithms for, 243–249

- vector form of, 244

- performance measures of, 263–267

- misadjustment as, 263–267

- time constants as, 263–264, 266–267

- performance surface of

- mean-squared error and, 242–244

- quadratic, 243, 247–248

- search algorithms for, 243–249

- prediction in, 255–258

- in real time, 241, 243

- recovery factors, 258

- time constants in

- for forgetting correlation estimate, 260

- for mean-squared error, 247–248

- as system performance measure, 263–264, 266–267

- for weight convergence, 246–247, 252, 255

- weights in, 243–249

- convergence factors, 246–247, 251–252, 255, 258, 263
- learning curve for, 247–249, 255
- LMS examples of, 255–258
- recursive least squares and, 258, 261–263
- system performance measures, 263–267
- Additive noise, in least-squares system design, 228
- Algebra
 - in linear systems, 79–80
 - vector and matrix, using MATLAB notation, 5–11
 - in window formulas, for finite impulse response filters, 119–121
- Algorithm(s)
 - for adaptive signal processing
 - direct descent, 258–263
 - least-mean-square, 241, 249, 254–258
 - one-step, 245
 - other, 267–268
 - random search, 243
 - recursive least squares, 258–263
 - steepest-descent, 249–254, 258
 - for Fast Fourier transform, 45–48, 99–103
 - Levinson's, 206
 - for linear systems, 87–103
 - direct, 87–89, 91
 - fast Fourier transform, 99–103
 - lattice, 91–99
 - state-space, 89–91
 - for mean-squared error minimization, 205–206
- Aliasing
 - in least-squares system design, 201–203
 - with multirate signal decomposition, 300, 302–304, 306
 - of signal spectrum, 39, 65–66
 - cross-power, 192–193
 - prevention of, 181–182
- Allpass filter
 - in finite impulse response, 122
 - in infinite impulse response, 160, 162
- Amplitude distribution(s)
 - continuous vs. discrete, 168–170
 - of random signals
 - introduction to, 167–168
 - principles of, 168–171
- Amplitude response (gain)
 - of finite impulse response differentiator, 127–128
 - of infinite impulse response filters, 138, 157–162
 - in linear systems, 77–79
- Amplitude spectrum
 - estimation of, *see* Spectral estimation
 - of signal vector, 48–51
- Analog-digital conversion
 - in finite impulse response filters, 115–116
 - in infinite impulse response filters, 147–149
 - bilinear transformation for, 149–153
 - of random signals, 169
- Analog prefiltering, 181–182
- Arithmetic coding, 287–291
 - adaptive, 290
- Array
 - adaptive, 267–268
 - image, 5
 - index, in mean-squared error minimization, 205
 - in MATLAB language, 3–5
- Array operations
 - in algebra notation, 10–11
 - summary of, 6
- Array products
 - in fading operation, 8–9
 - in linear discrete system, 90
 - types of, 7–8
- Autocorrelation, 40
 - in cross-power spectrum, 191–192
 - in least-squares system design, 213, 218, 229
 - of signal power spectrum, 181–183
- Autocorrelation matrix
 - in least-squares equations, 205
 - inverse, in adaptive signal processing, 258–260
 - normal form of, in adaptive signal processing, 251
- Autocorrelation vector, in mean-squared error minimization, 205
- Autocovariance matrix, in least-squares system design, 211–213
 - with broadband noise, 229–230
- Average cross-power, 191
- Average periodogram, in power spectral estimation, 185
- Average periodogram method, of power spectral estimation, 186–187
- Average power, of random stationary signals, 177–178, 181
 - example of, 181–182
- Average product, of two signals, 40

B

- Backslash, as algebraic operation, 11
- Bandpass filter
 - finite impulse response, 121–124
 - infinite impulse response, 135, 147–148, 153–154
- Bandstop filter
 - finite impulse response, 121–124
 - infinite impulse response, 135, 147–148, 153–154
- Bandwidth, in infinite impulse response frequency translation, 147–149
- Bar graph, of power density, 180
- Beamforming, adaptive, in signal processing, 268
- Bessel function, in Kaiser window, for finite impulse response filters, 121
- Bias weight, 232
- Bilinear transformation, for analog-digital conversion, in infinite impulse response filters, 149–153
- Binary code tree, in Huffman coding, 284–286
- Binary digit, *see* Bits
- Bits
 - coding of, 282; *see also* Coding
 - compression of, 277; *see also* Compression
 - as information unit, 278
 - two states of, 278
- Blackman window formula
 - for finite impulse response filters, 119–120
 - in spectral estimation, 190
- Blind equalization, in adaptive signal processing, 268
- Block adaptive process, 242
- Block fast Fourier transform filtering, in linear systems, 103
- Block processing, in least-squares system design, 210
- “Boats,” as image compression example, 296–297
- Boxcar window
 - for lowpass finite impulse response filters, 116, 118
 - in spectral estimation, 189–190
- Broadband noise, effects on least-squares system design, 228–230
- Butterworth filters, as infinite impulse response filters, 137–141
 - Chebyshev vs., 145–146
- Byte, as information unit, 278

C

- Cascade filter structure
 - in infinite impulse response filters, 139–140, 144, 148, 155–158, 162
 - of linear systems, 84–85
- Causality
 - of discrete linear systems, 72, 86
 - in finite impulse response filters, 114
- Channel noise, in least-squares system design, 228
- Characteristic equation, in steepest-descent algorithm, 250
- Chebyshev filters, as infinite impulse response filters, 141–146
 - Butterworth vs., 145–146
- Code lengths, in Huffman coding, 287
- Coding, of signals
 - adaptive, 290
 - amplitude distribution and, 176
 - arithmetic, 287–291
 - entropy, 281–291
 - equal-band, 306–307, 310
 - exercises for, 312–317
 - fixed-symbol, 283–287, 290
 - Huffman, 283–287, 290
 - linear predictive, 201, 224, 281, 298
 - multirate processing for, 291, 298–308
 - objective of, 277, 291
 - octave-band, 306–307, 310–312
 - predictive, 281–282, 292, 298
 - string, 282
 - subband, 298–299, 306–308
 - terms used with, 278
 - transform, 282, 291–298
 - vector quantization as, 282
- Coefficient(s)
 - in adaptive signal processing, 243–249
 - convergence factors, 246–247, 251–252, 255, 258, 263
 - learning curve for, 247–249, 255
 - LMS examples of, 255–258
 - recursive least squares and, 258, 261–263
 - in system performance measures, 263–267
 - in discrete linear systems, 72–73, 98–99, 101
 - in finite impulse response filters, 111–114, 121–124
 - formula for finding, 126
 - in infinite impulse response filters, 135, 141
 - analog-to-digital conversion of, 154–157

- in least-squares system design
 - as bias, 232
 - identification example, 218–219
 - for interference cancellation, 221, 223, 230, 232
 - for optimal mean-squared error minimization, 205–210
- Column vector
 - in algebra notation, 7
 - in least squares, 22
- Comb filter, infinite impulse response, 159–160
- Commands, in MATLAB language, 3
- Complex plane, description of, 12–13
- Complex products, in Fourier transforms, discrete vs. fast, 46
- Complex signals
 - finite impulse response filter processing of, 129
 - instantaneous power of, 177
- Components
 - low-frequency vs. high-frequency, in image compression, 296–298
 - in multirate signal decomposition, 307
- Compression, of signals
 - adaptive vs. nonadaptive, 256–258
 - amplitude distribution and, 176
 - entropy coding as, 281–291
 - exercises for, 312–317
 - with inverse transfer functions, 86, 224
 - linear predictor for, 201, 224–225, 227
 - multirate processing for, 291, 298–308
 - objective of, 277, 291
 - power spectrum and, 181
 - prediction error in, 281–282
 - transform coding as, 282, 291–298
 - two stages for, 280–282
- Compression ratio, in least-squares system design, 227
- Conjugate pairs
 - of fractions, in linear systems, 80, 84
 - of poles, in infinite impulse response filters, 139, 153–154
- Continuous amplitude distributions, 168–170
- Continuous form, of Fourier series, 30–32
- Continuous Fourier transform
 - in ideal lowpass filter, 112
 - inverse, 59, 113
 - of signal spectrum, 39, 58–60
- Continuous least-squares fit, 20
- Continuous probability function, in uniform random signals, 172
- Contour plot, in mean-squared error minimization, 208–209, 233
- Convergence, in adaptive signal processing
 - under ideal conditions, 246–249, 252
 - learning curve for weights, 247–248, 255
 - parameter for, 245–246
 - as performance measure, 263–264, 266–267
 - recursive least squares and, 261–263
 - steepest-descent algorithm for, 249, 251–254, 258
 - time constant for forgetting correlation estimate, 260
 - time constant for LMS, 254–255
 - time constant for MSE, 247–248
 - time constant for weights, 246–247
 - time constant under ideal conditions, 252
 - weight factors, 246–247, 251–252, 255, 258, 263
- Convolution
 - in least-squares system design, 212–213
 - of linear equations
 - discrete, 74–75
 - fast Fourier transform algorithms and, 99–103
 - periodic, 100–101
 - z-transform of, 75–76
 - in lowpass finite impulse response filters, 116–119
- Correlation
 - in adaptive signal processing, recursive estimate of, 258–260
 - in cross-power spectrum, 192
 - exercises for, 66–69
 - in least-squares equations, 205–207
 - broadband noise effects on, 229–230
 - computation of, 212–214
 - system identification example, 218–219
 - in signal processing, 39–42
- Correlation detector, 41–42
- Correlation estimate, recursive, in adaptive signal processing, 258–260
- Cosine(s)
 - basic applications of, 13
 - in Fourier series, 26, 28–29, 39
 - in infinite impulse response filters, 141–145
- Covariance function, in least-squares equations, 211–212

- computation of, 212–214
 - under noisy conditions, 229–230
 - system identification example, 218–219
- Cross-correlation function, 192
 - least-squares computations with, 212–213
 - broadband noise effects on, 229–230
- Cross-covariance, in least-squares system design
 - broadband noise effects on, 229–230
 - computations with, 213–214
 - equations for, 211
- Cross-periodogram, 191
- Cross-power density, 191
- Cross-power spectrum, 190–193
- Cross spectrum, 190–193
- Cutoff frequency, in infinite impulse response filters, 140
- D**
- Data files, in MATLAB, 3, 14, 312
- Data transformation, encryption as, 277–278
- Data windows, *see* Window entries
- DCT, *see* Discrete cosine transform (DCT)
- Decimation
 - in finite impulse response filters, 115–116
 - in multirate signal decomposition, 299–302, 312
- Decoding, entropy, with multirate signal decomposition, 299–300
- Decomposition, *see* Signal decomposition
- Decorrelation, in least-squares system design, 201
- Degrees of freedom
 - in power spectral estimation, 184
 - in sampled waveforms, 4
- Dependent variable, in discrete linear systems, 71–72
- Determinant, in algebra notation, 11
- DFT, *see* Discrete Fourier transform (DFT)
- Diagonal matrix, in adaptive signal processing, 250–253, 261
- Difference equations, 71
- Digital filters
 - discrete linear systems and, 103–104
 - as finite impulse response filters, 111, 129
 - as infinite impulse response filters, 137, 145
 - designing, 153–154
 - transformation to, 149–153
 - writing functions for, 153–157
- Digital resonator, for infinite impulse response filters, 158
- Digital signal processing (DSP)
 - adaptive, *see* Adaptive signal processing
 - algebra review for, 5–11
 - geometry review for, 12–14
 - introduction to, 1–2, 6
 - language for, *see* MATLAB
 - MATLAB functions in, 3, 14–15; *see also* Functions
 - text review techniques, 2, 15–16
- Direct algorithms, for linear systems, 87–89, 91
 - conversion to lattice, 94–95
 - lattice conversion to, 93–94
- Direct descent algorithm, for adaptive signal processing, 258–263
- Direct form, of linear systems, 87–89
- Direct-to-lattice conversion, in linear systems, 94–95
 - nonrecursive, 97–98
- Direction-finding system, 236–237
- Discrete amplitude distributions, 168–170
- Discrete convolution, of linear equations, 74–75
- Discrete cosine transform (DCT)
 - short-time, in signal analysis, 308–310
 - as signal compression, 292–298
 - discrete Fourier transform function with, 292–296
 - fast Fourier transform computation of, 294–295
 - inverse, 292–293, 295
 - multirate processing vs., 300, 306
 - two-dimensional, 295–298
- Discrete Fourier series
 - continuous form of, 30–32
 - exercises for, 33–36
 - fundamentals of, 26–27
 - as least-squares approximation, 27–30
 - MATLAB code implications, 30–31
 - summary of, 30–31
- Discrete Fourier transform (DFT)
 - complex products in, 46
 - continuous, 54, 58–60, 112
 - discrete cosine transform function of, 292–296
 - frequency increments in, 50

- interpolation of time-domain in, 53, 55, 63–64
- inverse, 51–52, 62
- in multirate signal decompression, 300–305
- principles of, 42–43
- properties of, 52–57
- redundancy in, 43–45, 52, 62
- resampling of, 52, 68
- in spectral analysis, 39, 43–44, 48–51
- z-transforms vs., 75–77
- Discrete least-squares process, 20, 22, 24
- Discrete linear equations
 - continuous systems vs., 71
 - convolution of, 74–75
 - properties of, 71–73
 - state-space, 89–91
- Discrete linear systems
 - causality of, 72, 86
 - coefficients in, 72–73, 98–99, 101
 - digital filters and, 103–104
 - realizability of, 72, 85–86
 - state-space equations for, 89–91
 - time domain in, 71, 73–75, 85
 - variables in, 71–72, 75–76
- Distortion, *see* Aliasing
- Distributed minimum of error surface, 210
- Down-sampling
 - in finite impulse response filters, 115–116
 - with multirate signal decomposition, 299–302, 310–312
- DSP, *see* Digital signal processing (DSP)
- E**
- EC, *see* Entropy coding (EC)
- ED (entropy decoding), with multirate signal decomposition, 299–300
- Eigenvalues, in adaptive signal processing, 250–253, 264, 266
- Eigenvector, in adaptive signal processing, 250
- Eigenvector matrix, in adaptive signal processing, 251
- Encoding, of signals, *see* Coding
- Encryption
 - as data transformation, 277–278
 - with inverse transfer functions, 86, 224
 - in least-squares system design, 224, 227
- Energy density, of signals, 178, 180
- Entropy, as information measure, 279
- Entropy coding (EC), as signal compression, 281–283
 - arithmetic scheme for, 287–291
 - Huffman scheme for, 283–287
 - in multirate decomposition, 299–300
- Entropy decoding (ED), with multirate signal decomposition, 299–300
- Envelope, exponential, imposed by array products, 9–10
- Equal-band coding, 306–307, 310
- Equal bands, in multirate signal decomposition, 306–307, 310
- Equalization
 - blind, in adaptive signal processing, 268
 - in least-squares system design, 199, 201–202
 - methods for, 215–217
- Equalizer, in least-squares system design, 215–216
- Equal-ripple property, of infinite impulse response filters, 141–142
- Equation error, for least-squares IIR system identification, 235–236
- Error(s)
 - quantizing, in sampling theorem, 61
 - total squared, in least squares, 19–21, 24
 - in waveform reconstruction, 64
- Error correction, in signals, 277
- Error detection, in signals, 277
- Error function, in Gaussian amplitude distributions, 174
- Error surface, distributed minimum of, 210
- Event, in information measurement, 279
- Expansion, with multirate signal decomposition, 302, 305
- Expected power, of stationary function, 177
- Expected value, in correlation, 40
- Exponential envelope, imposed by array products, 9–10
- Exponentiation, as algebraic operation, 11
- Expression simplification, geometric formulas for, 12–13
- Extension
 - periodic, in linear systems, 100
 - zero, of discrete Fourier transform, 52–54

F

Fading operation, for array products, 8–9

Fast Fourier transform (FFT)

- algorithms for, 45–48
- in linear systems, 99–103
- complex products in, 46
- discrete cosine transform
 - computation with, 294–295

- filtering in linear systems
 - block, 103

- nonrecursive, 101–102

- in multirate signal decomposition, 300, 306

Feedback control systems, 86–87

Feedback structure, of linear systems, 85–86

FFT, *see* Fast Fourier transform (FFT)

Filter(s)

- in adaptive signal processing, 242–244, 267–268

- in signal processing, 103, 157, 235, 268

Filter gain, *see* Power gain

Filtering

- in linear systems

- block fast Fourier transform, 103

- cascade, 84–85

- digital, 103–104, 111, 129

- finite, *see* Finite impulse response (FIR) filters

- infinite, *see* Infinite impulse response (IIR) filters

- inverse, 223–224

- lattice, 96, 98

- nonrecursive fast Fourier transform, 101–102

- power gain of, 103–104

- recursive vs. nonrecursive, 73

- multirate signal decomposition as, 298–305

Finite impulse response (FIR)

- differentiator, 126–128

Finite impulse response (FIR) filters, 111–134

- in adaptive signal processing, 242–244, 267–268

- advantages of, 111–112

- allpass, 122

- bandpass, 121–124

- bandstop, 121–124

- complete example of, 125–126

- digital, 111, 129

- disadvantages of, 111

- exercises for, 130–133

Hibert transformer as, 128–129

highpass, 121–124

introduction to, 73, 111–112

least-squares design with, 210–212, 235

linear phase of, 111–112, 114

lowpass

- conversion to other types, 121–124

- ideal characteristics of, 112–113

- window functions for, 116–121

in multirate signal decomposition, 302, 304–305

other types of, 126–129

passband ripple in, 114–115, 119, 121

power gain factors, 112, 114–115

- window spectrum as, 117–124

realizable version of, 113–116

stopband ripple in, 115, 119, 121

truncation of, 115, 117, 121

weights of, 111–114, 121–124

- formula for finding, 126

Finite signal vectors, in least-squares system design, 210–212

FIR (finite impulse response)

- differentiator, 126–128

FIR filters, *see* Finite impulse response (FIR) filters

Fixed-symbol coding, 283–287, 290

Floating-point operations, in fast Fourier transforms, 47–48

Flops, in fast Fourier transforms, 47–48

Forward transform, in multirate signal decomposition, 303

Fourier coefficients

- in correlation function, 42–43

- principles of, 28–31

Fourier Series

- discrete, *see* Discrete Fourier series

- harmonic analysis in, 13, 26, 28

- introduction to, 19

- reconstruction of signals, 62, 69

Fourier spectra, 39

- analysis of, 43, 48–51

- exercises for, 66–69

- frequency principles in, 43, 48–51

Fourier transforms (FT), of signal spectrum, 39

- analytic applications of, 48–51

- continuous, 54, 58–60, 112

- discrete, *see* Discrete Fourier transform (DFT)

- fast, *see* Fast Fourier transform (FFT)

- in ideal lowpass filter, 112–113

- inverse, 51–52, 59, 62, 113

- Fractions, in linear systems, 80–81, 84–85
 - Frequency and frequency domain
 - of amplitude distributions, 168, 175–176
 - in discrete linear systems, 71, 75–76
 - fundamental, in Fourier series, 27–29
 - as image compression factor, 296–297
 - of lowpass finite impulse response filters, 117
 - in multirate signal decomposition, 306–312
 - signal power density and, 177–178, 180–181
 - in signal processing, 39
 - Frequency conversion factors, in spectral analysis, 49
 - Frequency function, of digitized random signals, 169
 - Frequency increments, in spectral analysis, 50
 - Frequency resolution, of signal information, 308–310
 - Frequency scales, in spectral analysis, 48–50
 - Frequency translations, analog-to-digital
 - in finite impulse response filters, 115–116
 - in infinite impulse response filters, 147–149
 - bilinear transformation for, 149–153
 - Frequency units
 - in finite impulse response filters, 112
 - in signal power analysis, 180
 - FT, *see* Fourier transforms (FT)
 - Functions, MATLAB
 - a_decode*, 289–290
 - a_encode*, 289–290
 - angle*, 51
 - autocorr*, 213
 - autocorr_mat*, 213
 - autocovar_mat*, 214
 - bar2*, 194
 - bessel_0*, 121
 - bilin*, 154–155
 - bw_analog_weights*, 154–155
 - bw_lowpass_size*, 154
 - bw_weights*, 156
 - ch_analog_weights*, 154
 - ch_lowpass_size*, 154
 - ch_weights*, 156
 - code_length*, 287
 - contour*, 208–209, 233
 - conv*, 103
 - crosscorr*, 213
 - crosscovar*, 214
 - deconv*, 103
 - dir_to_lat*, 95
 - display_data*, 312
 - in DSP, 3, 14–15
 - erf*, 174
 - fft*, 47
 - fft2*, 296
 - fftshift*, 51, 55
 - filters*, 87, 103, 157
 - fir_weights*, 121, 123–124
 - flops*, 47–48
 - freq*, 193, 280, 287
 - h_codes*, 287
 - ifft*, 52
 - image_scan*, 297
 - imp_resp*, 103
 - index_mat*, 205
 - lat_filter*, 96
 - lat_to_dir*, 93
 - lms_filter*, 256, 262
 - nr_dir_to_lat*, 98
 - nr_lat_filter*, 98
 - nr_lat_to_dir*, 98
 - part_frac_exp*, 81–82
 - pds*, 194
 - pz_plot*, 81
 - rand*, 175
 - randn*, 175
 - reconst*, 63
 - resamp*, 64
 - residue*, 81
 - rls_filter*, 262
 - roots*, 80–81
 - row_vec*, 7, 15
 - sp_dct*, 295
 - sp_idct*, 295
 - unwrap*, 51
 - window*, 123
 - zig-zag_scan*, 297
 - Fundamental frequency, in Fourier series, 27–29
 - Fundamental period, in Fourier series, 27, 29
 - Fundamental terms, in Fourier series, 28
- ## G
- Gain curve, *see* Power gain
 - Gaussian distribution, of random signal
 - amplitude, 172–175
 - white noise in, 183, 186, 192, 217
 - Geometry
 - of adaptation, 241, 246
 - for expression simplification, 12–14

Gradient, in adaptive signal processing,
244, 252, 264
Gray-scale image, of amplitude
distributions, 175–176

H

Hanning window
for finite impulse response
differentiator, 127–128
for finite impulse response filters,
119–120
in spectral estimation, 190
Harmonic analysis, in Fourier series, 13,
26, 28
Harmonic functions, in Fourier series,
26–29
Help command, in MATLAB, 15
Hertz scale, in spectral analysis, 48–49
Hibert transformer, as finite impulse
response filters, 128–129
Highpass filters
with finite impulse response, 121–124
with infinite impulse response, 135,
147–149
in multirate signal decomposition,
299–302, 304–305
Huffman coding, 283–287, 290

I

Ideal conditions, in adaptive signal
processing, convergence under,
246–249, 252, 255
Identity matrix, definition of, 10
IIR filters, *see* Infinite impulse response
(IIR) filters
Image(s)
amplitude distribution of, 175–176
in MATLAB language, 5
mirror, in multirate signal
decomposition, 304–305
transform coding of, 291, 295–297
Image array, in MATLAB language, 5
Impulse response, in linear systems,
82–83, 100
Independent noise, effects on least-
squares system design, 228–230
Independent variable, in discrete linear
systems, 71–72
Index array, in mean-squared error
minimization, 205
Infinite impulse response (IIR) filters,
135–165
in adaptive signal processing, 243

allpass, 160, 162
bandpass, 135, 147–148, 153–155
bandstop, 135, 147–148, 153–154
bilinear transformation of, 149–153
Butterworth, 137–141
Chebyshev, 141–146
comb, 159–160
digital, 137, 145
designing, 153–154
transformation to, 149–153
writing functions for, 153–157
exercises for, 162–164, 235–236
highpass, 135, 147–149
introduction to, 73, 135
least-squares system identification
and, 235–236
linear phase of, 136–137
lowpass, 135, 137–141, 144
conversion to other types, 147–149,
153
other types of, 157–162
passband ripple in, 135–136, 140–142,
148, 156–157
power gain of
with analog-to-digital conversion,
151–156
Chebyshev vs. Butterworth,
145–146, 152
type-specific, 137–140, 143–145,
151–152, 159
spectrogram and, 160, 162
stopband ripple in, maximum gain in,
151–153
Infinite impulse response transfer
function, 135
linear phase of, 136–137
Information, *see* Signal information
Input signal, in linear systems, 72, 85,
136
Instantaneous power, of random
signals, 177
Integration by parts, 14
Interference canceling; *see also* Noise
in least-squares system design
broadband considerations for,
228–230
example of, 220–223, 232
principles of, 199, 202–203
Interpolation (time-domain), of discrete
Fourier transform, 53, 55, 63–64
Inverse, as algebraic operation, 10–11
Inverse autocorrelation, in adaptive
signal processing, 258–260
Inverse discrete cosine transform,
292–293, 295

- Inverse filtering, in least-squares system design, 223–224
- Inverse Fourier transforms
 - continuous, 59, 113
 - discrete, 51–52, 62
- Inverse modeling, least-squares system design for, 201–202
- Inverse transfer functions, signal compression with, 86, 224
- Inverse transform, in multirate signal decomposition, 303–304
- Inverse z-transform, linear systems and, 83–84
- Iteration, in Huffman coding, 285
- K**
- Kaiser window formula, for finite impulse response filters, 119–121, 123–124
- κ weights, of lattice structure, 98–99
- L**
- Laplace transfer function, in infinite impulse response filters, 138
- Laplace transform, 59, 83
- Lattice algorithms, for linear systems, 91–99
- Lattice filtering, in linear systems, 96, 98
- Lattice stage, symmetric two-multiplier, 91–92
- Lattice structure(s)
 - adaptive, 268
 - κ weights of, 98–99
 - symmetric
 - nonrecursive, 96–98
 - recursive, 91–92
- Lattice-to-direct conversion, in linear systems, 93–94
 - nonrecursive, 97–98
- Learning curve
 - for recursive least squares algorithm, 262–263
 - for weight convergence, in adaptive signal processing, 247–248, 255
- Least-mean-square adaptive predictor, 258
- Least-mean-square (LMS) algorithm, for adaptive signal processing
 - examples of, 255–258
 - performance measure for, 264–266
 - principles of, 241, 249, 254–255
 - recursive least squares algorithm vs., 261–262
- Least squares, 19–37
 - applications of, 20, 24, 26–30
 - in digital signal processing systems, 199–230
 - exercises for, 33–36
 - introduction to, 19
 - principle of, 19–24, 199
- Least-squares approximation, 20
 - discrete, 20, 22, 24
 - Fourier series as, 27–30
- Least-squares coefficients, in Fourier series, 28–29
- Least-squares coefficient vector, 24, 26
- Least-squares equalizer, for unknown channels, 215–216
- Least-squares equations
 - error identification in, 235–236
 - using correlation, 207
 - broadband noise effects on, 229–230
 - computation of, 212–214
 - system identification example, 218–219
 - using covariance, 211
- Least-squares polynomial, 20
- Least-squares predictor, for mean-squared error minimization, 207–208
- Least-squares system design, 199–239
 - applications of, 200–203
 - correlation in, 205–207
 - computation of, 212–214
 - covariance in, 211–212
 - computation of, 212–214
 - equalization in, 199, 201–202
 - of unknown channels, 215–216
 - example of, 207–210
 - exercises for, 230–237
 - with finite signal vectors, 210–212
 - interference canceling in
 - example of, 220–223, 232
 - principles of, 199, 202–203
 - introduction to, 19–24, 199
 - linear prediction and, 199–201
 - example of, 223–228
 - for mean-squared error minimization, 207–208
 - mean-squared error minimization in, 200, 203–207
 - illustration of, 207–210
 - modeling in, 199, 201–202, 217, 234, 237
 - noise in, 199, 201–203
 - effects of independent broadband, 228–230
 - improvement methods for, 220–223
 - system identification, 217–220

- "Lena," as amplitude distribution image, 175–176
 - Levinson's algorithm, for mean-squared error minimization, 206
 - Linear differential equations, 71
 - Linear equations
 - discrete vs. continuous, 71–72
 - solving simultaneous, 8
 - transfer functions and, 75–77
 - Linearity, of discrete Fourier transform, 52–53
 - Linear least-squares approximation, 20–22
 - Linear phase
 - of finite impulse response filters, 111–112, 114
 - of infinite impulse response filters, 136–137
 - Linear phase filter, 235
 - Linear phase shift
 - of discrete Fourier transform, 52, 56–57
 - by finite impulse response filters, 111–112, 114, 129
 - by infinite impulse response filters, 111, 136–137, 157, 162
 - in multirate signal decomposition, 305–306
 - principles of, 77, 79–80
 - Linear predictive coding, 201, 224, 281, 298
 - Linear predictor
 - compression with, 201, 224–225
 - encoding with, 201, 224
 - in least-squares system design, 199–201
 - example of, 223–228
 - for mean-squared error minimization, 207–208
 - recovery and, 223–228
 - Linear systems, 71–109
 - algorithms for, 87–103
 - direct, 87–89, 91
 - fast Fourier transform, 99–103
 - lattice, 91–99
 - state-space, 89–91
 - convolution of, 74–75
 - discrete, 74–75
 - fast Fourier transform algorithms and, 99–103
 - periodic, 100–101
 - z-transform of, 75–76
 - discrete vs. continuous, 71
 - exercises for, 105–108
 - poles and zeros in, 77–81, 86
 - processing speed of, 98–99
 - properties of, 71–72
 - structural illustrations of, 84–87
 - transient vs. stable, 81–84, 86, 112
 - z-transform and, 75–77
 - inverse, 83–84
 - Linear transfer functions, 75–77
 - exercises for, 105–108
 - fractions in, 80–81, 84–85
 - structure options for, 84–87
 - LMS algorithm, *see* Least-mean-square (LMS) algorithm
 - Log base 2, as information measure, 278–279
 - Lossless signal compression, 282, 291
 - with multirate decomposition, 300
 - Lossy signal compression, 282, 291
 - Lowpass filters
 - design of, 234–235
 - in finite impulse response
 - conversion to other types, 121–124
 - convolution of, 116–119
 - effects on power gain, 117–124
 - formulas for, 119–121
 - ideal characteristics of, 112–113
 - window functions for, 116–124
 - in infinite impulse response
 - conversion to other types, 147–149, 153
 - ideal characteristics of, 135, 137–141, 144
 - in multirate signal decomposition, 299–300, 302, 304–305
- ## M
- Magnitude, of discrete Fourier transform, 44
 - Magnitude-squared coherence (MSC), 191–193
 - Mathematical identities/operations; *see also specific type*
 - summary of, 13–14
 - MATLAB
 - in algebra notation, 5–11
 - basic language of, 2–3
 - element sequences processed by, 3–5
 - Fourier series implications for, 30–31
 - functions in DSP, *see* Functions
 - help* command, 15
 - introduction to, 2–3
 - utility of, 2
 - window formulas, for finite impulse response filters, 119–121
 - MATLAB data files, 3, 14, 312

MATLAB functions, *see* Functions

Matrix(ces)

algebra review of, 5–11

in least squares, 22–24

in MATLAB language, 3–5

Mean-squared error (MSE)

in adaptive signal processing

algorithm for, 241–242

gradient of, 244

least-mean-square convergence
with, 255–257

as performance measure, 263–267

performance surface of, 242–244

search algorithms for, 243–249

vector form of, 244

of least-squares process

applications of, 200, 203

broadband noise effects on, 229–230

equations for, 207

minimization algorithms, 205–206

minimization example, 207–210

system design via, 203–207

Mean-squared error gradient, as

adaptive signal processing

vector, 244

Mean-squared error plot, 208–209

Mean value, in amplitude distributions,
170–171

of Gaussian function, 173

of uniform variate, 172

m-files, in MATLAB language, 3, 14

Mirror image, in multirate signal
decomposition, 304–305

Modeling, in least-squares system

design, 199, 201–202, 217, 234, 237

MSC (magnitude-squared coherence),
191–193

MSE, *see* Mean-squared error (MSE)

Multiple-input adaptive systems, 268

Multirate processors, for signal coding
and compression, 302–303

Multirate signal decomposition, *see*

Multirate signal processing

Multirate signal processing, for coding
and compression

aliasing with, 300, 302–304

decimation/down-sampling with,
299–302, 310–312

decomposition with, 305–308

discrete cosine transform vs., 300, 306

discrete Fourier transform in, 300–305

as filtering process, 298–305

forward, 303

Fourier transform in, 300, 306

inverse, 303–304

principles of, 291, 298–299

recovery with, 299, 302–305

subband coding with, 298–299,
306–308

time-frequency resolution of, 310–311

wavelets in, 298–299

N

Neural networks, in adaptive signal
processing, 268

Newton's method, in adaptive signal
processing, 245

Node(s), in Huffman coding, 285

Noise

adaptive system performance and,
264–267

in cross-power spectrum, 192–193

in least-squares system design, 199,
201–203

effects of independent broadband,
228–230

improvement methods for, 220–223

Noise cancellation, with least-squares
design

example of, 220–223, 232

principles of, 199, 202–203

Noncausality, of discrete linear systems,
72

Nonlinear modeling, in least-squares
system, 237

Nonrectangular window, for lowpass
finite impulse response filters, 116

Nonrecursive fast Fourier transform
filtering, in linear systems,
101–102

Nonrecursive lattice, symmetric, 96–98

Nonrecursive linear equations, 73,

90–91, 101, 103; *see also* Finite
impulse response (FIR) filters

Nonsingular matrix, in algebra notation,
11

Nonstationary signal, 168

in adaptive signal processing, 243, 256

Normalized error function, in Gaussian
amplitude distributions, 174

Normal probability function, in
amplitude distributions, 172

O

Octave-band coding, 306–307, 310
wavelet transforms as, 310–312

Octave bands, in multirate signal
decomposition, 306–307, 311

- One, as bit state, 278
 - coding of, *see* Coding
- One-step algorithm, for adaptive signal processing, 245
- One-step predictor, for mean-squared error minimization, 207–208
 - in adaptive signal processing, 255–258
- Orthogonality, 19–37
 - exercises for, 33–36
 - in Fourier series, 28–30
 - introduction to, 19
 - principles of, 24–26
 - in steepest-descent algorithm, 250–251
- Output signal, in linear systems, 72, 85, 136
- Overlapping segments, in power spectral estimation, 185–189
- P**
- Parallel form, of transfer function, in linear systems, 84
- Parallel structure, of linear systems, 84–85
- Parameters
 - convergence, in adaptive signal processing, 245–246
 - for infinite impulse response filters, 140, 145, 149, 154
 - in linear systems
 - discrete, 73
 - for mean-squared error minimization, 203–204
- Parseval's theorem, 178
- Partial fractions, in linear systems, 81, 84
- Partitioning, in power spectral estimation, 185–186
- Passband ripple
 - in finite impulse response filters, 114–115, 119, 121
 - in infinite impulse response filters, 135–136, 140–142, 148, 156–157
- Performance feedback, in adaptation process, 242
- Performance surface
 - of mean-squared error
 - in adaptive signal processing, 242–244
 - search algorithms for, 243–249
 - quadratic, in adaptive signal processing, 243, 247–248
- Periodic convolution, of linear equations, 100–101
- Periodic extension, in linear systems, 100
- Periodic function, in Fourier series, 27
- Periodicity, of discrete Fourier transform, 52
- Periodogram
 - in cross-power spectrum, 191
 - power density and, 178–180
 - example of, 181–182
 - in power spectral estimation, 181–186
- Phase response (shift), in linear systems, 77, 79–80; *see also* Linear phase shift
- Phase spectrum, of signal vector, 48–51
- Phi*, in least-squares system design, 205–206, 213
- Pixel
 - coding and compression of, 278, 291, 296–298
 - in MATLAB language, 5
- Plant noise, in least-squares system design, 228
- Poles
 - in infinite impulse response filters, 135, 147, 158–159
 - with analog-to-digital conversion, 151–156
 - Butterworth, 138–139, 141, 149, 152
 - Chebyshev, 142–145, 152
 - conjugate pairs of, 139, 153–154
 - in linear systems, 77–81, 86
- Polynomial ratios/division
 - in filter systems, 111, 135–136, 141–142, 237
 - least-squares, 20
 - in linear systems, 99, 103
- Power density spectrum
 - of random signals
 - algorithms for, 193–194
 - cross-power, 190–193
 - data windows of, 187–190
 - estimation of, 184–188
 - introduction to, 167–168
 - principles of, 178–181
 - properties of, 181–183
 - white noise in, 183, 186, 192
- Power gain
 - of digital filters, 103–104, 151
 - of finite impulse response filters, 112, 114–115
 - window spectrum effects on, 117–124
 - of infinite impulse response filters
 - with analog-to-digital conversion, 151–156

- Chebyshev vs. Butterworth, 145–146, 152
 - individual types, 137–140, 143–145, 151–152, 159
 - Power spectral estimation
 - in adaptive signal processing, 261–262
 - principles of, 177–183
 - random signals and, 184–188
 - Power spectrum, *see* Power density spectrum
 - Prediction
 - in adaptive signal processing, 255–258
 - in least-squares system design, 199–201
 - for mean-square error minimization, 207–208
 - recovery and, 223–228
 - linear, of recovery, 224–228
 - of seismic events, 224–225
 - Prediction error
 - in adaptive signal processing, 257
 - in least-squares system design, 226
 - in signal compression, 281–282
 - Predictive coding, 281–282, 292, 298
 - linear, 201, 224, 281, 298
 - Prefiltered signals
 - for aliasing prevention, 181
 - in waveform reconstruction, 64–65
 - Prefix property, in Huffman coding, 285–286
 - Prewhitening, in least-squares system design, 227–228
 - Probability, in amplitude distributions, 169
 - Probability density function, of amplitude distributions, 169–171
 - Gaussian form of, 172–175
 - Probability function
 - continuous, in uniform random signals, 172
 - normal, in amplitude distributions, 172
 - Processing speed, of linear systems, 98–99
- Q**
- Quadratic error surface, in mean-squared error minimization, 210
 - Quadratic performance surface, in adaptive signal processing, 243, 247–248
 - Quadrature filter, 237
 - Quadrature mirror filters, 305
 - Quantization
 - with multirate signal decomposition, 299–300
 - vector, as coding, 282
 - Quantizing errors, in sampling theorem, 61
- R**
- Radian-per-second scale, in spectral analysis, 48–49
 - Random search algorithm, for adaptive signal processing, 243
 - Random signals, 167–198
 - amplitude distribution of
 - definition of, 167–168
 - principles of, 168–171
 - exercises for, 194–197
 - Gaussian distribution of, 172–175
 - introduction to, 167–168
 - other distributions of, 175–177
 - power density spectra of
 - algorithms for, 193–194
 - cross-power, 190–193
 - data windows of, 187–190
 - definition of, 167–168, 178
 - estimation of, 184–188
 - principles of, 178–181
 - properties of, 181–183
 - power of, 177–178
 - stationarity of, 167–169
 - statistical properties of, 167, 193–194
 - uniform distribution of, 171–172
 - Realizability
 - of discrete linear systems, 72, 85–86
 - of finite impulse response filters, 113–116
 - Real time
 - adaptive signal processing in, 241, 243
 - digital filters and phase shift in, 136
 - Reconstruction error, in least-squares system design, 226
 - Recovery
 - with compression
 - in adaptive signal processing, 258
 - in multirate signal decomposition, 299, 302–305
 - linear prediction of, 224–225
 - in least-squares system design, 223–228
 - Rectangular window
 - for lowpass filters
 - finite impulse response, 116, 118, 121

- infinite impulse response, 140, 145–146
 - in spectral estimation, 187–188, 190
 - Recursive adaptive filters, in signal processing, 268
 - Recursive estimate, in adaptive signal processing, 259
 - Recursive lattice, symmetric, 91–92
 - Recursive least squares (RLS) algorithm, for adaptive signal processing, 258–263
 - convergence with, 261–263
 - correlation estimates with, 258–260
 - forms of, 258, 261
 - learning curve for, 262–263
 - least-mean-square algorithm vs., 261–262
 - Recursive linear equations, 72, 102–103; *see also* Infinite impulse response (IIR) filters
 - Redundancy
 - of discrete Fourier transform, 43–45, 52, 62
 - removal of, 277–278
 - Resampling, of discrete Fourier transform, 52, 68
 - Residue Theorem, 83
 - Reversal
 - of signal vectors, in digital filters, 136–137
 - time series, in infinite impulse response filters, 137
 - RLS algorithm, *see* Recursive least squares (RLS) algorithm
 - Row vector, in algebra notation, 7, 15
- S**
- Sampled waveform(s), in MATLAB language, 4–5
 - Sample vector, in least squares, 20
 - Sampling frequency, in spectral analysis, 50
 - Sampling interval, in MATLAB language, 4
 - Sampling Theorem
 - exercises for, 66–69
 - Fourier transforms as proof of, 39, 54, 56, 61–62
 - simply stated, 60–61
 - in waveform reconstruction, 62–64
 - Scanning, in image compression, 297–298
 - Second harmonic terms, in Fourier series, 28
 - Segments
 - in multirate signal decomposition, 307–308
 - in power spectral estimation, 185 overlapping, 185–189
 - Seismic event(s)
 - adaptive signal processing of, 241
 - prediction and recovery of, 224–225
 - Short-time discrete transform, of signal information, 308–310
 - Signal component, in discrete Fourier transform, 56–57
 - Signal decomposition, with multirate signal processing, 305–308
 - Signal energy, 178, 180
 - Signal information, 277–318
 - coding, 282–308; *see also* Coding
 - compressing, 280–308; *see also* Compression
 - exercises for, 312–317
 - introduction to, 277–278
 - measuring, 278–280
 - time-frequency analysis of, 308–312
 - wavelet transforms of, 310–312
 - Signal power, *see* Power entries
 - Signal processing
 - adaptive, *see* Adaptive signal processing
 - digital, *see* Digital signal processing (DSP)
 - multirate, *see* Multirate signal processing
 - speed of
 - in adaptive systems, 263–264
 - in linear systems, 98–99
 - Signals
 - in linear systems, 72, 100
 - in MATLAB language, 4–5
 - reversal of, in digital filters, 136–137
 - Signal space
 - in MATLAB language, 4
 - orthogonal vectors in, 25
 - Signal-to-noise ratio (SNR)
 - in cross-power spectrum, 192–193
 - in least-squares system design, 201–203
 - effects of broadband, 228–230
 - improvement methods for, 220, 222–223
 - Signal vectors, *see* Vector(s)
 - Sine(s)
 - basic applications of, 10, 13–14
 - in Fourier series, 26, 28–29, 39
 - in least squares, 23
 - SNR, *see* Signal-to-noise ratio (SNR)

- Spatial domain, spectrum mapping of, 39
 - Spectral analysis
 - discrete Fourier transform in, 39, 43–44, 48–51
 - exercises for, 66–69
 - frequency principles in, 49–50
 - transform coding of, 295–298
 - white noise in, 183, 186, 192
 - Spectral estimation, 167–198
 - in adaptive signal processing, 261–262
 - algorithms for, 193–194
 - cross-power, 190–193
 - data windows in, 187–190
 - exercises for, 194–197
 - introduction to, 167–168
 - power principles of, 177–183, 261–262
 - techniques for, 184–188
 - Spectrogram
 - of infinite impulse response, 160–161
 - of signal information, 308–309
 - Spectrum, in signal processing, 39
 - Speech signal
 - compression of, 291, 295
 - in interference canceling, 221–222
 - multirate signal decomposition of, 307–308
 - Speed, of processing
 - in adaptive systems, 263–264
 - in linear systems, 98–99
 - s-plane, in infinite impulse response filters, 138–139
 - transformation to z-plane, 149–150
 - Stability, of linear systems, 81–83, 86
 - discrete, 73
 - infinite impulse response filters and, 136, 139
 - symmetric, 94
 - Standard deviation, in amplitude distributions, 170
 - of uniform variate, 172
 - Startup sequence, for discrete convolution, 74–75
 - Startup time, of finite impulse response filters, 112
 - State-space algorithms, for linear systems, 89–91
 - State-space equations, for discrete linear system, 89–91
 - State-space form, of linear systems, 89–91
 - State variable, of linear systems, 89
 - Stationarity, of signals, 167–169
 - Stationary function, instantaneous power of, 177–178
 - Stationary signal, 167–169
 - mean-squared error in, 204–205
 - total squared error in, 211
 - Statistical properties, of random signals, 167, 193–194
 - Steepest-descent algorithm, for adaptive signal processing, 249–254
 - convergence criterion for, 249, 251–254
 - eigenvalues of, 250–253, 264, 266
 - goals of, 249–250, 258
 - Stopband ripple
 - in finite impulse response filters, 115, 119, 121
 - in infinite impulse response filters, 135, 140–142, 145–146
 - maximum gain in, 151–153
 - String coding, 282
 - Subband coding, with multirate signal decomposition, 298–299, 306–308
 - Superscripts, in multirate signal decomposition, 307
 - Symbol(s)
 - as coding term, 278
 - in entropy coding, 279–281, 283
 - Symbol code, 278; *see also* Coding
 - Symbol entropy, 279
 - Symmetric nonrecursive lattice, 96–98
 - Symmetric recursive lattice, 91–92
 - Symmetric two-multiplier lattice stage, 91–92
 - System identification, least-squares system design for, 201
- ## T
- Tapered rectangular window, in spectral estimation, 190
 - Time constant, in adaptive signal processing
 - for forgetting correlation estimate, 260
 - for mean-squared error, 247–248
 - as system performance measure, 263–264, 266–267
 - for weight convergence, 246–247, 252, 255
 - Time domain
 - of discrete Fourier transform interpolation of, 53, 55, 63–64
 - zero extension in, 52–54
 - in discrete linear systems, 71, 73–75, 85
 - signal power density and, 178, 180
 - in signal processing, 39, 312

Digital Signal Processing with Examples in MATLAB[®]

Samuel D. Stearns

In a field as rapidly expanding as digital signal processing, even the basic topics change over time both in nature and relative importance. It is important, therefore, to have an up-to-date text that not only covers the fundamentals, but also follows a logical development that leaves no gaps that readers must somehow bridge by themselves.

Digital Signal Processing with Examples in MATLAB[®] is just such a text. The presentation does not focus on DSP in isolation, but relates it to continuous signal processing and treats digital signals as samples of physical phenomena. The author introduces important topics not usually addressed in signal processing texts, including the discrete cosine transform, multirate signal processing, signal coding and compression, least squares systems design, and adaptive signal processing. He also uses the industry-standard software MATLAB to provide DSP functions and examples.

Designed for a one-semester upper-level course but also ideal for self-study and reference, *Digital Signal Processing with Examples in MATLAB[®]* is complete, self-contained, and rigorous. For basic DSP, it is quite simply the only book you need.

FEATURES

- Offers an outstanding balance of DSP theory, techniques, and applications
- Provides a concise MATLAB tutorial and describes dozens of useful MATLAB functions
- Describes digital transforms and algorithms
- Covers digital filter analysis and design and least-squares system design
- Introduces statistical signal processing and spectral analysis
- Explores adaptive signal processing systems
- Furnishes all of the examples and functions used in the text, as well as exercise solutions, online at www.crcpress.com.

CRC PRESS
www.crcpress.com

1091

ISBN 0-8493-1091-1



9 780849 310911

